FAKULTET INFORMATIKE U PULI

FORMULA 1

DOKUMENTACIJA ZA PROJEKTNI ZADATAK

TIM 8

Krnjus Sebastian
Lakošeljac Rea
Levkova Mila
Meden Mia
Radola Leon
Vlk Nikolina

BAZE PODATAKA 2

Mentori:
Romeo Šajina
Goran Oreški

PULA, 30. SIJEČNJA 2025.

Sadržaj

1. UVOD 5
2. ER DIJAGRAM 6
Opis ER-dijagrama 6
3. EER dijagram - MYSQL WORKBENCH 9
4. OPIS TABLICA BAZE PODATAKA FORMULA 11
4.1. Tablica tim 11
4.2. Tablica bolid 12
4.3. Tablica vozac 13
4.4. Tablica vozac_bolid 14
4.5. Tablica sezona 14
4.6. Tablica staza 15
4.7. Tablica utrka 16
4.8. Tablica sudjelovanje 17
4.9. Tablica rezultat 18
4.10. Tablica strategija 19
4.11. Tablica kazna 19
4.12. Tablica poredak 20
4.13. Tablica sponsor 21
4.14. Tablica ugovor 22
4.15. Tablica osoblje 23
4.16. Tablica zaduzenje 23
4.17. Tablica trosak_tima 24
4.18. Tablica korisnici 25
Tablica: sistemski_log (TEMPORARY) 25
5. UPITI 26
5.1. Rezultati jedne utrke 26
5.2. Svi rezultati vozača 26
5.3. Prikaz bodova u sezoni 27
5.4. Vozači koji su u sezoni osvojili bodove u više od jedne utrke 28
5.5. Vozači bez ijedne kazne u sezoni 29
6. POGLEDI 30
6.1. Poredak sezone 30

6.2 Statistika vozača u sezoni 31
6.3 Statistički podaci o sustavu 32
6.4 Statistika timova 33
6.5 Ukupni broj bodova za timove 34
6.6 Najbrži vozači 34
6.7 Broj utrka po sezoni 35
6.8 Prosječna starost vozača 36
6.9 Analiza uspješnosti timova po budžetu 37
6.10 Poredak utrke 38
6.11 Vozači po najvećim brojem kaznenih bodova u sezoni 39
6.12 Broj utrka u državama 40
7. PROCEDURE 41
7.1. Izračunavanje poretka sezone 41
7.2. Najbrži vozač tima 42
7.3. Popis naziva staza 43
7.4. Dodavanje utrke 44
7.5. Povećanje broja pobjeda vozaču 45
7.6. Prosječna kazna po utrkama za tim 46
8. FUNKCIJE 48
8.1. Bodovi vozača u sezoni 48
8.2. Pretvaranje brzine 49
8.3. Sudjelovanja vozača u utrkama 50
8.4. Vozač s najvećim brojem nastupa 51
8.5. Najbolji vozač u timu 52
8.6. Prosječne kaznene sekunde vozača po utrci u sezoni 53
8.7. Postotak sprint utrka u sezoni za vozača 54
9. OKIDAČI 56
9.1. Sprječavanje promjene naziva tima 56
9.2. Automatsko dodavanje kazne na vrijeme 57
9.3. Automatsko dodavanje bodova 58
9.4. Kontrola budžeta tima pri unosu novog troška 60
9.5. Provjera dobi vozača 61
9.6. Praćenje budžeta tima 61

9.7. Provjera prijave vozača 62
10. TRANSAKCIJE 64
10.1 Unos podataka o pobjedniku i ažuriranje poretka 64
10.2 Brisanje bodova za zasluženu kaznu 65
10.3 Promjena stanja bolida 66
10.4 Evidentiranje prelaska vozača u drugi tim 67
10.5 Poništavanje plaćanja ako tim prekorači budžet 68
11. INICIJALIZACIJA I KONFIGURACIJA 70
12. RUTE I FUNKCIONALNOSTI 70
13. INTERAKCIJA S BAZOM 70
14. PORUKE I REDIREKCIJA 71

1. UVOD

U ovoj dokumentaciji predstavljen je projekt iz kolegija Baze podataka 2 pod nazivom **„Formula 1“**. Cilj projekta je izrada baze podataka koja služi za pohranu i obradu podataka vezanih uz automobilske utrke. Baza obuhvaća najvažnije dijelove ovog sporta, kao što su timovi, vozači, bolidi, staze, utrke, sponzori i ostalo osoblje.

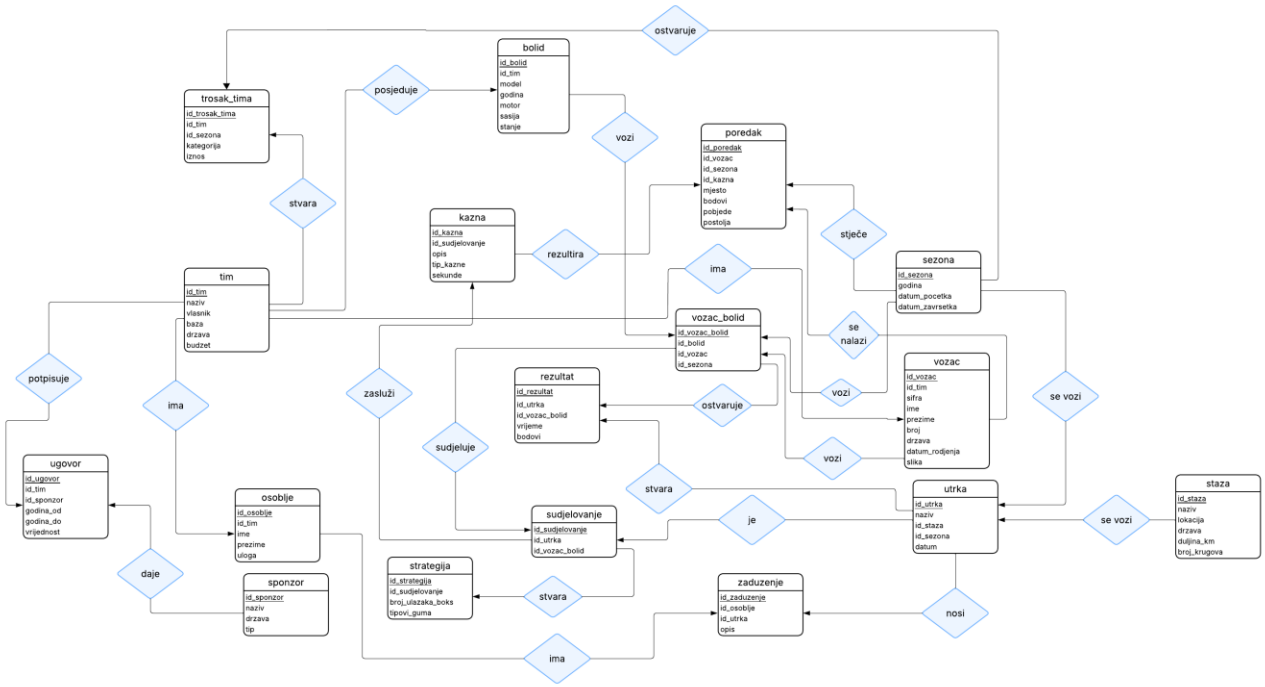
Projekt je zamišljen kao sustav koji omogućuje jednostavno upravljanje i pregled podataka vezanih uz automobilizam. Baza podataka omogućuje praćenje rezultata utrka, sezona, kazni, strategija, troškova timova i drugih važnih informacija. Na taj način moguće je dobiti cjelovit uvid u natjecanja i uspješnost vozača i timova.

U dokumentaciji je poseban naglasak stavljen na tehnički dio projekta. Opisan je način izrade tablica, veza između njih te korištenje SQL naredbi. Također su prikazane funkcije, procedure, okidači i transakcije koji osiguravaju ispravan rad baze podataka, točnost podataka i pouzdanost sustava.



Slika 1 Slika je generirana pomoću umjetne inteligencije

2. ER DIJAGRAM



Opis ER-dijagrama

ER dijagram prikazuje strukturu baze podataka sustava za upravljanje automobilističkim natjecanjima. Svaki pravokutnik u dijagramu predstavlja entitet odnosno tablicu u bazi podataka, dok linije između entiteta označavaju relacije i logičke veze među njima. Dijagram jasno definira organizaciju podataka, njihove međusobne odnose te ograničenja koja osiguravaju integritet sustava.

Ključni entiteti i njihovi atributi

Tim
Entitet *Tim* predstavlja natjecateljske timove koji sudjeluju u sezoni. Sadrži atribute poput jedinstvenog identifikatora, naziva tima, vlasnika, baze tima, države te budžeta. Budžet je ograničen uvjetom da ne može imati negativnu vrijednost. Ovaj entitet ima središnju ulogu u sustavu jer je povezan s vozačima, bolidima, osobljem, sponzorima i troškovima.

Bolid
Entitet *Bolid* sadrži podatke o vozilima koja koriste timovi. Atributi uključuju identifikator bolida, model, godinu proizvodnje, motor, šasiju i stanje bolida. Svaki bolid je povezan s jednim timom, a stanje bolida ograničeno je na unaprijed definirane vrijednosti poput „Ispravan“, „Na servisu“ ili „Otpisan“.

Vozač
Entitet *Vozač* sadrži osobne i sportske podatke o vozačima. Atributi uključuju jedinstveni

identifikator, šifru, ime, prezime, startni broj, državu, datum rođenja te pripadajući tim. Dodatno su definirana ograničenja koja osiguravaju jedinstvenost startnog broja unutar tima te jedinstvenu kombinaciju šifre i prezimena.

Sezona
Entitet *Sezona* predstavlja natjecateljsku sezonu, primjerice sezonu 2025. Sadrži podatke o godini te datumima početka i završetka sezone, uz ograničenje koje osigurava logičan redoslijed datuma.

Vozač–Bolid
Entitet *vozac_bolid* predstavlja veznu tablicu koja povezuje vozače i bolide po sezonama. Ovaj entitet omogućuje praćenje koji je vozač vozio koji bolid u određenoj sezoni, čime se rješava veza tipa više-na-više između vozača i bolida.

Staza
Entitet *Staza* opisuje trkaće staze na kojima se utrke održavaju. Sadrži atribute poput naziva, lokacije, države, duljine staze i broja krugova. Broj krugova mora biti veći od nule, što osigurava realne podatke.

Utrka
Entitet *Utrka* predstavlja pojedinačni natjecateljski događaj. Svaka utrka povezana je s jednom stazom i jednom sezonom te sadrži podatke o nazivu i datumu održavanja. Ovaj entitet ima ključnu ulogu jer povezuje rezultate, sudjelovanja, kazne i strategije.

Sudjelovanje
Entitet *Sudjelovanje* predstavlja prijavu vozača (odnosno kombinacije vozač–bolid) na određenu utrku. Ova tablica osigurava da se rezultati i kazne mogu unositi samo za vozače koji su službeno sudjelovali u utrci.

Rezultat
Entitet *Rezultat* sadrži podatke o ishodu utrke za pojedinog vozača. Atributi uključuju vrijeme, osvojene bodove te poveznice na utrku i vozača–bolid. Ograničenje osigurava da broj bodova ne može biti negativan.

Strategija
Entitet *Strategija* bilježi strategiju vozača tijekom utrke, uključujući broj ulazaka u boks i korištene vrste guma. Svaka strategija vezana je uz točno jedno sudjelovanje u utrci.

Kazna
Entitet *Kazna* sadrži informacije o kaznama dodijeljenim tijekom utrke. Atributi uključuju opis, tip kazne i kaznene sekunde. Tip kazne ograničen je na unaprijed definirani skup dopuštenih vrijednosti.

Poredak
Entitet *Poredak* predstavlja ukupni poredak vozača u određenoj sezoni. Sadrži podatke o mjestu, bodovima, broju pobjeda, broju postolja te eventualnoj kazni. Ovaj entitet omogućuje praćenje uspješnosti vozača kroz cijelu sezonu.

Sponzor
Entitet *Sponzor* sadrži podatke o sponzorima, uključujući naziv, državu i tip sponzorstva.

Ugovor
Entitet *Ugovor* predstavlja odnos između timova i sponzora. Sadrži podatke o trajanju ugovora i njegovoj vrijednosti, čime se omogućuje praćenje financijskih odnosa.

Osoblje
Entitet *Osoblje* sadrži podatke o članovima tima, poput mehaničara, inženjera i ostalog osoblja. Svaki član osoblja pripada jednom timu.

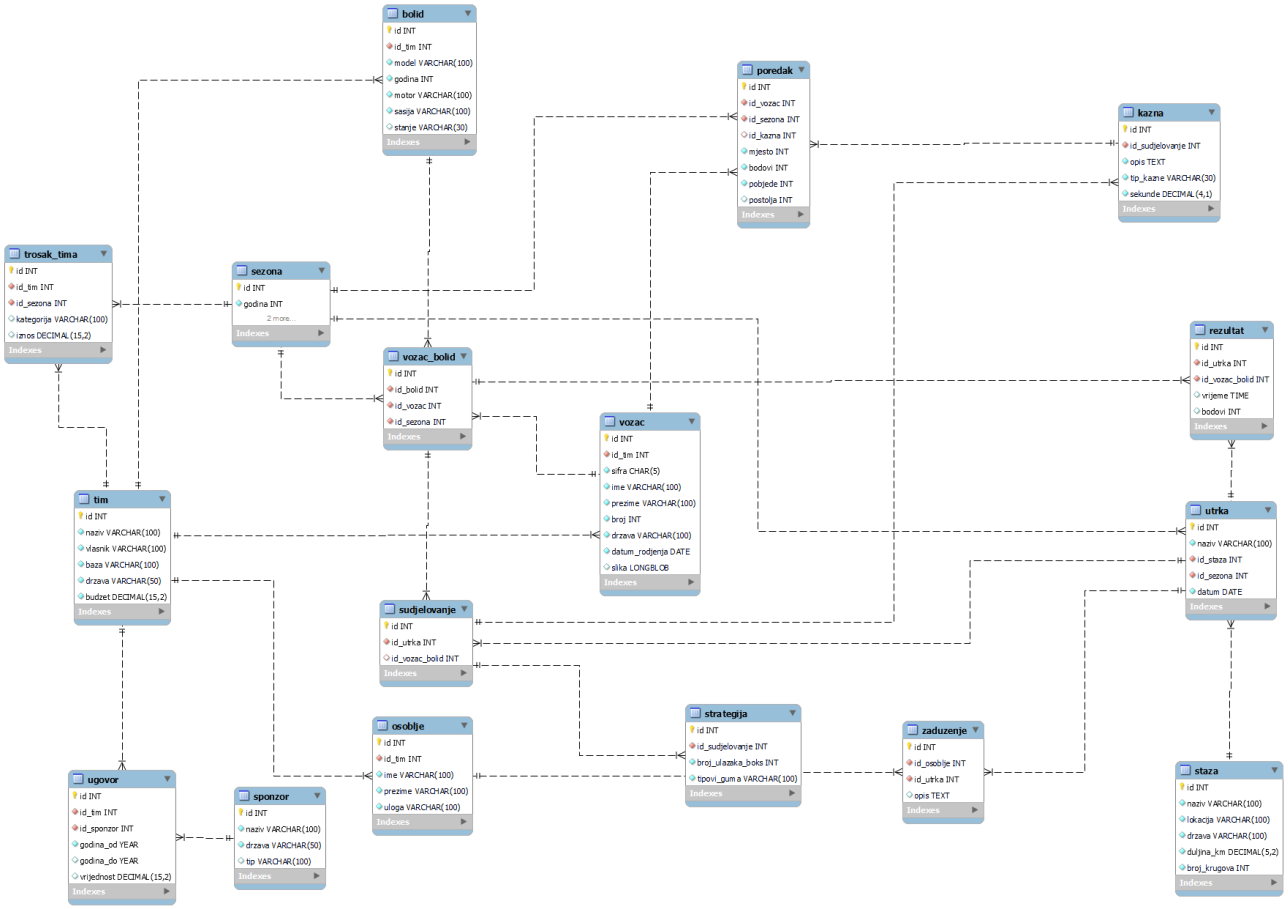
Zaduženje
Entitet *Zaduženje* opisuje zadatke koje osoblje ima na pojedinoj utrci, čime se omogućuje praćenje aktivnosti osoblja po utrkama.

Trošak tima
Entitet *trosak_tima* bilježi financijske troškove timova po sezoni, uključujući kategoriju troška i iznos. Ograničenje osigurava da iznos ne može biti negativan.

Veze i kardinaliteti

Veze između entiteta jasno definiraju odnose u sustavu. Primjerice, jedan tim može imati više vozača, bolida i članova osoblja, dok svaki vozač pripada točno jednom timu. Utrka se odvija na jednoj stazi i pripada jednoj sezoni, ali sezona može imati više utrka. Veze su implementirane pomoću stranih ključeva, čime se osigurava referencijalni integritet baze.

3. EER dijagram - MYSQL WORKBENCH



4. OPIS TABLICA BAZE PODATAKA FORMULA

4.1. Tablica tim

```sql
CREATE TABLE tim (
    id INT AUTO_INCREMENT PRIMARY KEY,
    naziv VARCHAR(100) NOT NULL UNIQUE,
    vlasnik VARCHAR(100) NOT NULL,
    baza VARCHAR(100) NOT NULL,
    drzava VARCHAR(50) NOT NULL,
    budzet DECIMAL(15,2) NOT NULL CHECK (budzet >= 0)
);
```

Sadrži osnovne podatke o Formula 1 timovima te njihov financijski budžet.

id: Jedinstveni identifikacijski broj tima, koji se automatski povećava za svaki novi zapis (PRIMARY KEY, AUTO_INCREMENT).

naziv: Naziv tima, maksimalno 100 znakova. Stupac mora biti popunjen (NOT NULL) i jedinstven (UNIQUE).

vlasnik: Naziv vlasnika tima, maksimalno 100 znakova. Stupac mora biti popunjen (NOT NULL).

baza: Grad u kojem se nalazi glavna baza tima, maksimalno 100 znakova. Stupac mora biti popunjen (NOT NULL).

drzava: Država u kojoj je tim registriran, maksimalno 50 znakova. Stupac mora biti popunjen (NOT NULL).

budzet: Financijski budžet tima izražen kao decimalni broj. Stupac mora biti popunjen (NOT NULL), a vrijednost ne smije biti negativna (CHECK budzet >= 0).

4.2. Tablica bolid

```
CREATE TABLE bolid (
    id INT AUTO_INCREMENT PRIMARY KEY,
    id_tim INT NOT NULL,
    model VARCHAR(100) NOT NULL,
    godina INT NOT NULL,
    motor VARCHAR(100) NOT NULL,
    sasija VARCHAR(100) NOT NULL,
    stanje VARCHAR(30) DEFAULT 'Ispravan',
    CONSTRAINT chk_stanje CHECK (stanje IN ('Ispravan', 'Neispravan', 'Na servisu', 'Otpisan')
    FOREIGN KEY (id_tim) REFERENCES tim(id)
        ON DELETE CASCADE
        ON UPDATE CASCADE
);
```

Sadrži podatke o bolidima koje posjeduju timovi.

id: Jedinstveni identifikacijski broj bolida, koji se automatski povećava za svaki novi zapis (PRIMARY KEY, AUTO_INCREMENT).

id_tim: Identifikacijski broj tima kojem bolid pripada. Stupac mora biti popunjen (NOT NULL).

model: Naziv modela bolida, maksimalno 100 znakova. Stupac mora biti popunjen (NOT NULL).

godina: Godina proizvodnje bolida, predstavljena kao cijeli broj. Stupac mora biti popunjen (NOT NULL).

motor: Naziv ili oznaka motora bolida, maksimalno 100 znakova. Stupac mora biti popunjen (NOT NULL).

sasija: Oznaka šasije bolida, maksimalno 100 znakova. Stupac mora biti popunjen (NOT NULL).

stanje: Trenutno stanje bolida. Dozvoljene vrijednosti su: Ispravan, Neispravan, Na servisu, Otpisan. Zadana vrijednost je 'Ispravan'.

FOREIGN KEY povezuje id_tim sa stupcem id u tablici tim, osiguravajući da svaki bolid pripada postojećem timu. Pravila ON DELETE CASCADE i ON UPDATE CASCADE osiguravaju automatsko brisanje ili ažuriranje povezanih zapisa.

4.3. Tablica vozac

```
CREATE TABLE vozac (
    id INT AUTO_INCREMENT PRIMARY KEY,
    id_tim INT NOT NULL,
    sifra CHAR(5) NOT NULL UNIQUE,
    ime VARCHAR(100) NOT NULL,
    prezime VARCHAR(100) NOT NULL,
    broj INT NOT NULL,
    drzava VARCHAR(100) NOT NULL,
    datum_rodjenja DATE NOT NULL,
    slika LONGBLOB,
    FOREIGN KEY (id_tim) REFERENCES tim(id)
        ON DELETE CASCADE
        ON UPDATE CASCADE,
    CONSTRAINT uniq_broj_tim UNIQUE(id_tim, broj),
    CONSTRAINT uniq_sifra_prezime UNIQUE (sifra, prezime)
);
```

Sadrži osobne podatke o vozačima i povezuje ih s timom.

id: Jedinstveni identifikacijski broj vozača, koji se automatski povećava za svaki novi zapis (PRIMARY KEY, AUTO_INCREMENT).

id_tim: Identifikacijski broj tima kojem vozač pripada. Stupac mora biti popunjen (NOT NULL).

sifra: Jedinstvena šifra vozača (npr. VER01). Stupac mora biti popunjen (NOT NULL) i jedinstven (UNIQUE).

ime: Ime vozača, maksimalno 100 znakova. Stupac mora biti popunjen (NOT NULL).

prezime: Prezime vozača, maksimalno 100 znakova. Stupac mora biti popunjen (NOT NULL).

broj: Startni broj vozača unutar tima. Stupac mora biti popunjen (NOT NULL), a kombinacija (id_tim, broj) mora biti jedinstvena (UNIQUE).

drzava: Država iz koje vozač dolazi, maksimalno 100 znakova. Stupac mora biti popunjen (NOT NULL).

datum_rodjenja: Datum rođenja vozača. Stupac mora biti popunjen (NOT NULL). Sustav putem okidača ne dopušta unos vozača mlađeg od 18 godina.

slika: Slika vozača pohranjena u binarnom obliku (LONGBLOB). Stupac nije obavezan.

FOREIGN KEY povezuje id_tim sa stupcem id u tablici tim, osiguravajući da svaki vozač pripada postojećem timu.

4.4. Tablica vozac_bolid

```
CREATE TABLE vozac_bolid (
    id INT AUTO_INCREMENT PRIMARY KEY,
    id_bolid INT NOT NULL,
    id_vozac INT NOT NULL,
    id_sezona INT NOT NULL,
    FOREIGN KEY (id_bolid) REFERENCES bolid(id)
        ON DELETE CASCADE
        ON UPDATE CASCADE,
    FOREIGN KEY (id_vozac) REFERENCES vozac(id)
        ON DELETE CASCADE
        ON UPDATE CASCADE,
    FOREIGN KEY (id_sezona) REFERENCES sezona(id)
        ON DELETE CASCADE
        ON UPDATE CASCADE
);
```

Tablica koja veže vozača i bolid za određenu sezonu (omogućuje promjene po sezonama).

id: Jedinstveni identifikacijski broj zapisa, koji se automatski povećava za svaki novi zapis (PRIMARY KEY, AUTO_INCREMENT).

id_bolid: Identifikacijski broj bolida koji vozač koristi. Stupac mora biti popunjen (NOT NULL).

id_vozac: Identifikacijski broj vozača koji koristi bolid. Stupac mora biti popunjen (NOT NULL).

id_sezona: Identifikacijski broj sezone u kojoj vozač koristi određeni bolid. Stupac mora biti popunjen (NOT NULL).

FOREIGN KEY povezuje id_bolid sa stupcem id u tablici bolid, osiguravajući da zapis odgovara postojećem bolidu.

FOREIGN KEY povezuje id_vozac sa stupcem id u tablici vozac, osiguravajući da zapis odgovara postojećem vozaču.

FOREIGN KEY povezuje id_sezona sa stupcem id u tablici sezona, osiguravajući da je kombinacija važeća za određenu sezonu.

4.5. Tablica sezona

```
CREATE TABLE sezona ( #specifično 2025
    id INT AUTO_INCREMENT PRIMARY KEY,
    godina INT NOT NULL,
    datum_pocetka DATE,
    datum_zavrsetka DATE
);
```

Definira natjecateljske sezone

id: Jedinstveni identifikacijski broj sezone, koji se automatski povećava za svaki novi zapis (PRIMARY KEY, AUTO_INCREMENT).

godina: Godina održavanja sezone, predstavljena kao cijeli broj. Stupac mora biti popunjen (NOT NULL).

datum_pocetka: Datum početka sezone. Stupac nije obavezan.

datum_zavrsetka: Datum završetka sezone. Stupac nije obavezan.

4.6. Tablica staza

```
CREATE TABLE staza (
    id INT AUTO_INCREMENT PRIMARY KEY,
    naziv VARCHAR(100) NOT NULL UNIQUE,
    lokacija VARCHAR(100) NOT NULL,
    drzava VARCHAR(100) NOT NULL,
    duljina_km DECIMAL(5,2) NOT NULL,
    broj_krugova INT NOT NULL,
    CONSTRAINT chk_broj_krugova CHECK (broj_krugova > 0)
);
```

Sadrži podatke o stazama na kojima se voze utrke.

id: Jedinstveni identifikacijski broj staze, koji se automatski povećava za svaki novi zapis (PRIMARY KEY, AUTO_INCREMENT).

naziv: Naziv staze, maksimalno 100 znakova. Stupac mora biti popunjen (NOT NULL) i jedinstven (UNIQUE).

lokacija: Lokacija ili grad u kojem se staza nalazi, maksimalno 100 znakova. Stupac mora biti popunjen (NOT NULL).

drzava: Država u kojoj se staza nalazi, maksimalno 100 znakova. Stupac mora biti popunjen (NOT NULL).

duljina_km: Duljina jednog kruga staze izražena u kilometrima. Stupac mora biti popunjen (NOT NULL).

broj_krugova: Broj krugova koji se voze na utrci. Stupac mora biti popunjen (NOT NULL) i veći od 0 (CHECK broj_krugova > 0).

4.7. Tablica utrka

```
CREATE TABLE utrka (
    id INT AUTO_INCREMENT PRIMARY KEY,
    naziv VARCHAR(100) NOT NULL,
    id_staza INT NOT NULL, #gdje
    id_sezona INT NOT NULL, #ime natjecanja
    datum DATE NOT NULL,
    FOREIGN KEY (id_staza) REFERENCES staza(id)
        ON DELETE CASCADE
        ON UPDATE CASCADE,
    FOREIGN KEY (id_sezona) REFERENCES sezona(id)
        ON DELETE CASCADE
        ON UPDATE CASCADE

);
```

Sadrži podatke o utrkama u sezoni i povezuje ih sa stazom.

id: Jedinstveni identifikacijski broj utrke, koji se automatski povećava za svaki novi zapis (PRIMARY KEY, AUTO_INCREMENT).

naziv: Naziv utrke, maksimalno 100 znakova. Stupac mora biti popunjen (NOT NULL).

id_staza: Identifikacijski broj staze na kojoj se utrka održava. Stupac mora biti popunjen (NOT NULL).

id_sezona: Identifikacijski broj sezone u kojoj se utrka održava. Stupac mora biti popunjen (NOT NULL).

datum: Datum održavanja utrke. Stupac mora biti popunjen (NOT NULL).

status: Status utrke (Najavljena, Traje, Zavrsena). Zadana vrijednost je 'Najavljena'.

FOREIGN KEY povezuje id_staza sa stupcem id u tablici staza, osiguravajući da svaka utrka bude povezana s postojećom stazom.

FOREIGN KEY povezuje id_sezona sa stupcem id u tablici sezona, osiguravajući da svaka utrka pripada određenoj sezoni.

4.8. Tablica sudjelovanje

```
CREATE TABLE sudjelovanje (
    id INT AUTO_INCREMENT PRIMARY KEY,
    id_utrka INT NOT NULL,
    id_vozac_bolid INT,
    FOREIGN KEY (id_utrka) REFERENCES utrka(id)
        ON DELETE CASCADE
        ON UPDATE CASCADE,
    FOREIGN KEY (id_vozac_bolid) REFERENCES vozac_bolid(id)
        ON DELETE CASCADE
        ON UPDATE CASCADE,
    CONSTRAINT uniq_utrka_vozac_bolid UNIQUE(id_utrka, id_vozac_bolid)
);
```

Evidencija prijave (sudjelovanja) vozača/bolida na određenoj utrci.

id: Jedinstveni identifikacijski broj sudjelovanja, koji se automatski povećava za svaki novi zapis (PRIMARY KEY, AUTO_INCREMENT).

id_utrka: Identifikacijski broj utrke na kojoj vozač sudjeluje. Stupac mora biti popunjen (NOT NULL).

id_vozac_bolid: Identifikacijski broj kombinacije vozač–bolid. Stupac mora biti popunjen (NOT NULL).

Kombinacija (id_utrka, id_vozac_bolid) mora biti jedinstvena (UNIQUE), čime se sprječava višestruko sudjelovanje istog vozača u istoj utrci.

FOREIGN KEY povezuje id_utrka sa stupcem id u tablici utrka.

FOREIGN KEY povezuje id_vozac_bolid sa stupcem id u tablici vozac_bolid.

4.9. Tablica rezultat

```
CREATE TABLE rezultat (
    id INT AUTO_INCREMENT PRIMARY KEY,
    id_utrka INT NOT NULL, #koja utrka
    id_vozac_bolid INT NOT NULL, #tko je pobijedio
    vrijeme TIME,
    bodovi INT,
    CONSTRAINT chk_bodovi CHECK (bodovi >= 0),
    FOREIGN KEY (id_utrka) REFERENCES utrka(id)
        ON DELETE CASCADE
        ON UPDATE CASCADE,
    FOREIGN KEY (id_vozac_bolid) REFERENCES vozac_bolid(id)
        ON DELETE CASCADE
        ON UPDATE CASCADE
);
```

Pohranjuje rezultate vozača na utrci: vrijeme i osvojene bodove.

id: Jedinstveni identifikacijski broj rezultata, koji se automatski povećava za svaki novi zapis (PRIMARY KEY, AUTO_INCREMENT).

id_utrka: Identifikacijski broj utrke na kojoj je rezultat ostvaren. Stupac mora biti popunjen (NOT NULL).

id_vozac_bolid: Identifikacijski broj kombinacije vozač–bolid koja je ostvarila rezultat. Stupac mora biti popunjen (NOT NULL).

vrijeme: Ukupno vrijeme vožnje na utrci, predstavljeno kao vremenski zapis (TIME).

bodovi: Broj bodova osvojenih na utrci, predstavljen kao cijeli broj. Vrijednost ne smije biti negativna (CHECK bodovi >= 0).

FOREIGN KEY povezuje id_utrka sa stupcem id u tablici utrka.

FOREIGN KEY povezuje id_vozac_bolid sa stupcem id u tablici vozac_bolid.

4.10. Tablica strategija

```
CREATE TABLE strategija (
    id INT AUTO_INCREMENT PRIMARY KEY,
    id_sudjelovanje INT NOT NULL,
    broj_ulazaka_boks INT NOT NULL,
    tipovi_guma VARCHAR(100) NOT NULL,
    CONSTRAINT chk_ulasci_boks CHECK (broj_ulazaka_boks >= 0),
    FOREIGN KEY (id_sudjelovanje) REFERENCES sudjelovanje(id)
        ON DELETE CASCADE
        ON UPDATE CASCADE
);
```

Opis strategije tijekom utrke (boksovi, gume) za određeno sudjelovanje.

id: Jedinstveni identifikacijski broj strategije, koji se automatski povećava za svaki novi zapis (PRIMARY KEY, AUTO_INCREMENT).

id_sudjelovanje: Identifikacijski broj sudjelovanja na koje se strategija odnosi. Stupac mora biti popunjen (NOT NULL).

broj_ulazaka_boks: Broj ulazaka u boks tijekom utrke. Stupac mora biti popunjen (NOT NULL) i ne smije biti negativan.

tipovi_guma: Opis korištenih tipova guma tijekom utrke. Stupac mora biti popunjen (NOT NULL).

FOREIGN KEY povezuje id_sudjelovanje sa stupcem id u tablici sudjelovanje.

4.11. Tablica kazna

```
CREATE TABLE kazna (
    id INT AUTO_INCREMENT PRIMARY KEY,
    id_sudjelovanje INT NOT NULL,
    opis TEXT NOT NULL,
    tip_kazne VARCHAR(30) NOT NULL,
    sekunde DECIMAL(4,1) NOT NULL,
    CONSTRAINT chk_sek CHECK (sekunde >= 0),
    CONSTRAINT chk_tip_kazne CHECK (tip_kazne in ('Time Penalty', 'Drive-through penalty',
    'Stop-and-go penalty', 'Time added after race','Grid penalty',
    'Black flag', 'Disqualification', 'Warning', 'Penalty points' )),
    FOREIGN KEY (id_sudjelovanje) REFERENCES sudjelovanje(id)
        ON DELETE CASCADE
        ON UPDATE CASCADE
);
```

Evidencija kazni dodijeljenih za određeno sudjelovanje na utrci.

id: Jedinstveni identifikacijski broj kazne, koji se automatski povećava za svaki novi zapis (PRIMARY KEY, AUTO_INCREMENT).

id_sudjelovanje: Identifikacijski broj sudjelovanja na koje se kazna odnosi. Stupac mora biti popunjen (NOT NULL).

opis: Tekstualni opis kazne. Stupac mora biti popunjen (NOT NULL).

tip_kazne: Vrsta kazne (npr. Time Penalty, Drive-through, Disqualification). Stupac mora biti popunjen (NOT NULL).

sekunde: Broj kaznenih sekundi. Vrijednost ne smije biti negativna.

FOREIGN KEY povezuje id_sudjelovanje sa stupcem id u tablici sudjelovanje.

4.12. Tablica poredak

```
CREATE TABLE poredak (
    id INT AUTO_INCREMENT PRIMARY KEY,
    id_vozac INT NOT NULL,
    id_sezona INT NOT NULL,
    id_kazna INT,
    mjesto INT NOT NULL ,
    bodovi INT NOT NULL DEFAULT 0,
    pobjede INT NOT NULL DEFAULT 0,
    postolja INT DEFAULT 0,
    CONSTRAINT chk_mjesto CHECK (mjesto > 0),
    FOREIGN KEY (id_vozac) REFERENCES vozac(id)
        ON DELETE CASCADE
        ON UPDATE CASCADE,
    FOREIGN KEY (id_sezona) REFERENCES sezona(id)
        ON DELETE CASCADE
        ON UPDATE CASCADE,
    FOREIGN KEY (id_kazna) REFERENCES kazna(id)
        ON DELETE CASCADE
        ON UPDATE CASCADE
);
```

Ukupni poredak vozača u određenoj sezoni (bodovi, pobjede, postolja, mjesto).

id: Jedinstveni identifikacijski broj zapisa poretka, koji se automatski povećava za svaki novi zapis (PRIMARY KEY, AUTO_INCREMENT).

id_vozac: Identifikacijski broj vozača. Stupac mora biti popunjen (NOT NULL).

id_sezona: Identifikacijski broj sezone. Stupac mora biti popunjen (NOT NULL).

id_kazna: Identifikacijski broj kazne, ako postoji. Stupac nije obavezan.

mjesto: Konačno mjesto vozača u sezoni. Vrijednost mora biti veća od 0.

bodovi: Ukupan broj bodova osvojenih u sezoni.

pobjede: Broj pobjeda vozača u sezoni.

postolja: Broj osvojenih postolja u sezoni.

FOREIGN KEY povezuje id_vozac sa stupcem id u tablici vozac.

FOREIGN KEY povezuje id_sezona sa stupcem id u tablici sezona.

FOREIGN KEY povezuje id_kazna sa stupcem id u tablici kazna.

4.13. Tablica sponsor

```
CREATE TABLE sponzor (
    id INT AUTO_INCREMENT PRIMARY KEY,
    naziv VARCHAR(100) NOT NULL,
    drzava VARCHAR(50) NOT NULL,
    tip VARCHAR(100)
);
```

Sadrži podatke o sponzorima (npr. Oracle, Shell).

id: Jedinstveni identifikacijski broj sponzora, koji se automatski povećava za svaki novi zapis (PRIMARY KEY, AUTO_INCREMENT).

naziv: Naziv sponzora, maksimalno 100 znakova. Stupac mora biti popunjen (NOT NULL).

drzava: Država sponzora, maksimalno 50 znakova. Stupac mora biti popunjen (NOT NULL).

tip: Tip ili djelatnost sponzora. Stupac nije obavezan.

4.14. Tablica ugovor

```
CREATE TABLE ugovor (
    id INT AUTO_INCREMENT PRIMARY KEY,
    id_tim INT NOT NULL,
    id_sponzor INT NOT NULL,
    godina_od YEAR NOT NULL,
    godina_do YEAR,
    vrijednost DECIMAL(15,2),
    CONSTRAINT chk_vrijendost  CHECK (vrijednost >= 0),
    FOREIGN KEY (id_tim) REFERENCES tim(id)
        ON DELETE CASCADE
        ON UPDATE CASCADE,
    FOREIGN KEY (id_sponzor) REFERENCES sponzor(id)
        ON DELETE CASCADE
        ON UPDATE CASCADE
);
```

Povezuje timove i sponzore te definira trajanje i vrijednost sponzorskog ugovora.

id: Jedinstveni identifikacijski broj ugovora, koji se automatski povećava za svaki novi zapis (PRIMARY KEY, AUTO_INCREMENT).

id_tim: Identifikacijski broj tima koji sklapa ugovor. Stupac mora biti popunjen (NOT NULL).

id_sponzor: Identifikacijski broj sponzora. Stupac mora biti popunjen (NOT NULL).

godina_od: Početna godina trajanja ugovora. Stupac mora biti popunjen (NOT NULL).

godina_do: Završna godina trajanja ugovora. Stupac nije obavezan.

vrijednost: Financijska vrijednost ugovora. Vrijednost ne smije biti negativna.

FOREIGN KEY povezuje id_tim sa stupcem id u tablici tim.

FOREIGN KEY povezuje id_sponzor sa stupcem id u tablici sponzor.

4.15. Tablica osoblje

```
CREATE TABLE osoblje (
    id INT AUTO_INCREMENT PRIMARY KEY,
    id_tim INT NOT NULL,
    ime VARCHAR(100)NOT NULL,
    prezime VARCHAR(100) NOT NULL,
    uloga VARCHAR(100) NOT NULL,
    FOREIGN KEY (id_tim) REFERENCES tim(id)
        ON DELETE CASCADE
        ON UPDATE CASCADE
);
```

Sadrži podatke o članovima osoblja timova (npr. Team Principal, Technical Director).

id: Jedinstveni identifikacijski broj osoblja, koji se automatski povećava za svaki novi zapis (PRIMARY KEY, AUTO_INCREMENT).

id_tim: Identifikacijski broj tima kojem osoblje pripada. Stupac mora biti popunjen (NOT NULL).

ime: Ime člana osoblja. Stupac mora biti popunjen (NOT NULL).

prezime: Prezime člana osoblja. Stupac mora biti popunjen (NOT NULL).

uloga: Uloga člana osoblja u timu. Stupac mora biti popunjen (NOT NULL).

FOREIGN KEY povezuje id_tim sa stupcem id u tablici tim.

4.16. Tablica zaduzenje

```
CREATE TABLE zaduzenje (
    id INT AUTO_INCREMENT PRIMARY KEY,
    id_osoblje INT NOT NULL,
    id_utrka INT NOT NULL,
    opis TEXT,
    FOREIGN KEY (id_osoblje) REFERENCES osoblje(id)
        ON DELETE CASCADE
        ON UPDATE CASCADE,
    FOREIGN KEY (id_utrka) REFERENCES utrka(id)
        ON DELETE CASCADE
        ON UPDATE CASCADE
);
```

Evidencija zaduženja osoblja na određenim utrkama (npr. strategija, setup bolida).

id: Jedinstveni identifikacijski broj zaduženja, koji se automatski povećava za svaki novi zapis (PRIMARY KEY, AUTO_INCREMENT).

id_osoblje: Identifikacijski broj člana osoblja. Stupac mora biti popunjen (NOT NULL).

id_utrka: Identifikacijski broj utrke. Stupac mora biti popunjen (NOT NULL).

opis: Opis zaduženja osoblja na utrci. Stupac nije obavezan.

FOREIGN KEY povezuje id_osoblje sa stupcem id u tablici osoblje.

FOREIGN KEY povezuje id_utrka sa stupcem id u tablici utrka.

4.17. Tablica trosak_tima

```
CREATE TABLE trosak_tima (
    id INT AUTO_INCREMENT PRIMARY KEY,
    id_tim INT NOT NULL,
    id_sezona INT NOT NULL,
    kategorija VARCHAR(100),
    iznos DECIMAL(15,2),
    CONSTRAINT chk_iznos CHECK (iznos >= 0),
    FOREIGN KEY (id_tim) REFERENCES tim(id)
        ON DELETE CASCADE
        ON UPDATE CASCADE,
    FOREIGN KEY (id_sezona) REFERENCES sezona(id)
        ON DELETE CASCADE
        ON UPDATE CASCADE
);
```

Evidencija troškova timova po sezoni (razvoj, motor, logistika itd.).

id: Jedinstveni identifikacijski broj troška, koji se automatski povećava za svaki novi zapis (PRIMARY KEY, AUTO_INCREMENT).

id_tim: Identifikacijski broj tima na koji se trošak odnosi. Stupac mora biti popunjen (NOT NULL).

id_sezona: Identifikacijski broj sezone. Stupac mora biti popunjen (NOT NULL).

kategorija: Kategorija troška (npr. razvoj, logistika). Stupac nije obavezan.

iznos: Iznos troška. Vrijednost ne smije biti negativna.

FOREIGN KEY povezuje id_tim sa stupcem id u tablici tim.

FOREIGN KEY povezuje id_sezona sa stupcem id u tablici sezona.

4.18. Tablica korisnici

```
-- Tablica za web login
CREATE TABLE korisnici (
    id INT AUTO_INCREMENT PRIMARY KEY,
    username VARCHAR(50) NOT NULL UNIQUE,
    password VARCHAR(50) NOT NULL,
    rola ENUM('admin', 'user') NOT NULL
);
```

Tablica za web prijavu (login) s korisničkim imenom, lozinkom i rolom.

id: Jedinstveni identifikacijski broj korisnika, koji se automatski povećava za svaki novi zapis (PRIMARY KEY, AUTO_INCREMENT).

username: Korisničko ime. Stupac mora biti popunjen (NOT NULL) i jedinstven (UNIQUE).

password: Lozinka korisnika. Stupac mora biti popunjen (NOT NULL).

rola: Uloga korisnika u sustavu (admin ili user). Stupac mora biti popunjen (NOT NULL).

Tablica: sistemski_log (TEMPORARY)

```
CREATE TEMPORARY TABLE sistemski_log (
    id INT AUTO_INCREMENT PRIMARY KEY,
    tablica VARCHAR(50),
    akcija VARCHAR(50),
    poruka TEXT,
    vrijeme TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

Privremena tablica za logiranje promjena tijekom sesije. Postoji samo dok traje sesija.

id: Jedinstveni identifikator log zapisa (AUTO_INCREMENT, PRIMARY KEY).

tablica: Naziv tablice na koju se akcija odnosi (VARCHAR(50)).

akcija: Vrsta akcije (npr. UPDATE) (VARCHAR(50)).

poruka: Detaljna poruka loga (TEXT).

vrijeme: Vrijeme zapisa (TIMESTAMP). Zadano CURRENT_TIMESTAMP.

5. UPITI

5.1. Rezultati jedne utrke

```
-- rezultati jedne utrke
SELECT * FROM rezultat WHERE id_utrka = 5;
```

Ovaj SQL upit služi za dohvat svih rezultata jedne konkretne utrke na temelju njezinog identifikacijskog broja. U upitu se iz tablice rezultat izdvajaju svi zapisi kod kojih vrijednost stupca id_utrka odgovara vrijednosti 5, što predstavlja identifikator odabrane utrke. Rezultat izvođenja upita je popis svih vozača koji su sudjelovali u toj utrci, zajedno s njihovim vremenima vožnje i osvojenim bodovima. Ovakav upit omogućuje jednostavan i brz pregled kompletnih rezultata pojedine utrke te se koristi za analizu plasmana, bodovanja i performansi vozača na specifičnoj utrci.

5.2. Svi rezultati vozača

```
-- svi rezultati vozača
SELECT * FROM rezultat WHERE id_vozac_bolid = 8;
```

Ovaj SQL upit služi za dohvat svih rezultata koje je ostvario određeni vozač kroz sve utrke u kojima je sudjelovao. Upit izdvaja zapise iz tablice rezultat za koje je vrijednost stupca id_vozac_bolid jednaka 8, pri čemu id_vozac_bolid predstavlja jedinstvenu kombinaciju vozača i bolida u sezoni. Rezultat izvođenja upita je popis svih utrka u kojima je taj vozač nastupio, zajedno s pripadajućim vremenima vožnje i brojem osvojenih bodova. Ovakav upit omogućuje analizu kontinuiteta performansi vozača, usporedbu rezultata po utrkama te praćenje ukupnog učinka vozača tijekom sezone.

5.3. Prikaz bodova u sezoni

```sql
-- bodovi u sezoni
SELECT SUM(bodovi)
FROM rezultat r
JOIN utrka u ON r.id_utrka = u.id
WHERE u.id_sezona = 1
  AND r.id_vozac_bolid = 8;
CREATE INDEX idx_rezultat_utrka
ON rezultat (id_utrka);

CREATE INDEX idx_rezultat_vozac
ON rezultat (id_vozac_bolid);

CREATE INDEX idx_rezultat_utrka_vozac
ON rezultat (id_utrka, id_vozac_bolid);

show index from rezultat;
```

Ovaj SQL upit služi za izračun ukupnog broja bodova koje je određeni vozač osvojio u jednoj sezoni. Upit koristi agregacijsku funkciju SUM nad stupcem bodovi iz tablice rezultat, čime se zbrajaju svi bodovi ostvareni na pojedinim utrkama. Tablica rezultat povezuje se s tablicom utrka putem vanjskog ključa id_utrka, kako bi se moglo filtrirati samo one utrke koje pripadaju određenoj sezoni. Uvjetom u.id_sezona = 1 ograničavaju se rezultati na sezonu s identifikacijskim brojem 1, dok se dodatnim uvjetom r.id_vozac_bolid = 8 osigurava da se bodovi računaju isključivo za odabranog vozača (odnosno njegovu kombinaciju vozač–bolid). Rezultat izvođenja ovog upita je jedan broj koji predstavlja ukupan broj bodova koje je vozač ostvario u toj sezoni, što omogućuje jednostavnu analizu učinka vozača na razini cijele sezone.

Indeks idx_rezultat_utrka kreiran je nad stupcem id_utrka u tablici rezultat s ciljem ubrzavanja upita koji filtriraju ili povezuju rezultate prema utrci. Budući da se stupac id_utrka često koristi u JOIN operacijama s tablicom utrka, ovaj indeks značajno poboljšava performanse takvih upita.

Indeks idx_rezultat_vozac kreiran je nad stupcem id_vozac_bolid u tablici rezultat. Njegova svrha je optimizacija upita koji dohvaćaju rezultate za određenog vozača, primjerice pri analizi svih nastupa i bodova pojedinog vozača tijekom sezone ili karijere.

Složeni indeks idx_rezultat_utrka_vozac kreiran je nad kombinacijom stupaca id_utrka i id_vozac_bolid. Ovaj indeks dodatno poboljšava performanse upita koji istovremeno filtriraju rezultate po utrci i po vozaču, što je čest slučaj u analitičkim upitima, poput provjere sudjelovanja vozača na određenoj utrci ili izračuna bodova za konkretnu kombinaciju vozač–bolid i utrka.

Upit SHOW INDEX FROM rezultat koristi se za prikaz svih indeksa definiranih nad tablicom rezultat. Njegovim izvođenjem dobiva se pregled naziva indeksa, stupaca koji su njima

obuhvaćeni, vrste indeksa te redoslijeda stupaca unutar složenih indeksa. Ovaj upit služi za provjeru ispravnosti i postojanja indeksa te za analizu optimizacije performansi baze podataka.

5.4. Vozači koji su u sezoni osvojili bodove u više od jedne utrke

```sql
SELECT
    vozac.ime AS ime_vozaca,
    vozac.prezime AS prezime_vozaca,
    tim.naziv AS ime_tima,
    sezona.godina AS sezona,
    COUNT(rezultat.id) AS broj_utrka_sa_bodovima,
    AVG(rezultat.bodovi) AS prosjek_bodova
FROM vozac
JOIN vozac_bolid ON vozac.id = vozac_bolid.id_vozac
JOIN sudjelovanje ON vozac_bolid.id = sudjelovanje.id_vozac_bolid
JOIN rezultat ON sudjelovanje.id = rezultat.id
JOIN utrka ON rezultat.id_utrka = utrka.id
JOIN sezona ON utrka.id_sezona = sezona.id
JOIN tim ON vozac.id_tim = tim.id
WHERE rezultat.bodovi > 0
GROUP BY vozac.id, sezona.id
HAVING COUNT(rezultat.id) > 1
ORDER BY prosjek_bodova DESC;
```

Ovaj SQL upit služi za identifikaciju vozača koji su u jednoj sezoni osvojili bodove u više od jedne utrke te za izračun njihovog prosječnog broja bodova po utrci. Upit povezuje podatke iz više tablica kako bi se dobio cjelovit kontekst o vozaču, njegovom timu, sezoni u kojoj nastupa te ostvarenim rezultatima.

U upitu se tablica vozac povezuje s tablicom vozac_bolid, čime se osigurava veza između vozača i njihovih nastupa u sezoni. Nadalje, preko tablice sudjelovanje povezuju se prijave vozača na pojedine utrke, dok tablica rezultat omogućuje dohvat osvojenih bodova. Tablice utrka i sezona koriste se za određivanje sezone u kojoj su bodovi ostvareni, a tablica tim omogućuje prikaz naziva tima kojem vozač pripada.

Uvjetom rezultat.bodovi > 0 filtriraju se samo oni nastupi u kojima je vozač osvojio barem jedan bod, čime se isključuju utrke bez bodovnog učinka. Grupiranje podataka provodi se po identifikacijskom broju vozača i sezone, što omogućuje agregaciju podataka na razini jednog vozača u jednoj sezoni. U HAVING dijelu upita dodatno se ograničava rezultat na vozače koji su bodove osvojili u više od jedne utrke, odnosno kod kojih je broj bodovnih nastupa veći od jedan.

Rezultat upita prikazuje ime i prezime vozača, naziv tima, godinu sezone, ukupan broj utrka u kojima je vozač osvojio bodove te prosječan broj bodova po utrci. Podaci su na kraju sortirani silazno prema prosječnom broju bodova, čime se vozači s najboljim prosječnim učinkom prikazuju na vrhu rezultata. Ovakav upit omogućuje usporedbu konzistentnosti i učinkovitosti vozača tijekom sezone.

5.5. Vozači bez ijedne kazne u sezoni

```
SELECT vozac.sifra, vozac.ime, vozac.prezime
    FROM vozac
    WHERE vozac.id NOT IN (
        SELECT vozac_bolid.id_vozac
            FROM kazna
                JOIN sudjelovanje ON sudjelovanje.id = kazna.id_sudjelovanje
                JOIN vozac_bolid ON vozac_bolid.id = sudjelovanje.id_vozac_bolid
                JOIN utrka ON utrka.id = sudjelovanje.id_utrka
                WHERE utrka.id_sezona = 1
);
```

Ovaj upit služi za dohvat vozača koji tijekom određene sezone nisu dobili nijednu kaznu. Cilj upita je identificirati vozače s besprijekornim ponašanjem tijekom natjecanja.

Upit iz tablice vozac dohvaća šifru, ime i prezime vozača, ali isključuje one vozače koji se pojavljuju u podupitu. Podupit pronalazi sve vozače koji su u zadanoj sezoni imali barem jednu kaznu, pri čemu se povezuju tablice kazna, sudjelovanje, vozac_bolid i utrka kako bi se osiguralo da se kazne odnose na točno određene vozače i sezonu.

Korištenjem uvjeta NOT IN u glavnom upitu osigurava se da se u konačnom rezultatu prikažu samo oni vozači koji nemaju nijedan zapis o kazni u odabranoj sezoni.

6. POGLEDI

6.1. Poredak sezone

```
CREATE OR REPLACE VIEW view_poredak_sezone AS
SELECT
    vb.id_vozac,
    u.id_sezona,
    SUM(r.bodovi) AS bodovi,
    SUM(CASE WHEN r.bodovi = 25 THEN 1 ELSE 0 END) AS pobjede,
    SUM(CASE WHEN r.bodovi >= 15 THEN 1 ELSE 0 END) AS postolja,
    DENSE_RANK() OVER (
        PARTITION BY u.id_sezona
        ORDER BY SUM(r.bodovi) DESC
    ) AS mjesto
FROM vozac_bolid vb
JOIN rezultat r ON r.id_vozac_bolid = vb.id
JOIN utrka u ON u.id = r.id_utrka
GROUP BY vb.id_vozac, u.id_sezona;
```

Pogled view_poredak_sezone je kreiran kako bi omogućio izračun i prikaz ukupnog poretka vozača unutar pojedine sezone na temelju ostvarenih rezultata u utrkama. Ovaj pogled povezuje podatke iz tablica vozac_bolid, rezultat i utrka, čime se ostvaruje veza između vozača, njihovih rezultata na utrkama i sezone u kojoj su ti rezultati ostvareni.

Za svakog vozača i sezonu prikazuje identifikacijski broj vozača (id_vozac) te identifikacijski broj sezone (id_sezona), zajedno s ukupnim brojem osvojenih bodova koji se računa kao zbroj svih bodova osvojenih u utrkama te sezone. Također se izračunava broj pobjeda, pri čemu se kao pobjeda broji svaka utrka u kojoj je vozač osvojio 25 bodova, te broj postolja, gdje se postoljem smatra svaka utrka u kojoj je vozač osvojio 15 ili više bodova.

Za određivanje konačne pozicije vozača u sezoni koristi se analitička funkcija DENSE_RANK, koja rangira vozače unutar iste sezone prema ukupnom broju osvojenih bodova, pri čemu vozači s istim brojem bodova dijele isto mjesto u poretku. Podaci su grupirani po vozaču i sezoni, čime se osigurava da svaki zapis u pogledu predstavlja jednog vozača u jednoj sezoni.

6.2 Statistika vozača u sezoni

```sql
DROP VIEW IF EXISTS statistika_vozaca_po_sezoni;

CREATE VIEW statistika_vozaca_po_sezoni AS
SELECT
    vozac.id AS vozac_id,
    vozac.ime AS ime_vozaca,
    vozac.prezime AS prezime_vozaca,
    tim.id AS tim_id,
    tim.naziv AS ime_tima,
    sezona.godina AS sezona,
    COUNT(DISTINCT utrka.id) AS broj_utrka,
    SUM(rezultat.bodovi) AS ukupni_bodovi,
    SUM(IF(rezultat.bodovi = 25, 1, 0)) AS broj_pobjeda,
    SUM(IF(rezultat.bodovi >= 15, 1, 0)) AS broj_postolja,
    SUM(kazna.sekunde) AS ukupne_kazne_sekunde,
    AVG(TIME_TO_SEC(rezultat.vrijeme) / staza.broj_krugova) AS prosjecno_vrijeme_po_krugu
FROM vozac
JOIN vozac_bolid ON vozac.id = vozac_bolid.id_vozac
JOIN sudjelovanje ON vozac_bolid.id = sudjelovanje.id_vozac_bolid
JOIN rezultat ON sudjelovanje.id = rezultat.id
JOIN utrka ON rezultat.id_utrka = utrka.id
JOIN staza ON utrka.id_staza = staza.id
JOIN sezona ON utrka.id_sezona = sezona.id
JOIN tim ON vozac.id_tim = tim.id
LEFT JOIN kazna ON sudjelovanje.id = kazna.id_sudjelovanje
GROUP BY vozac.id, sezona.id, tim.id
ORDER BY ukupni_bodovi DESC, broj_pobjeda DESC;

 SELECT * FROM statistika_vozaca_po_sezoni;
 SELECT * FROM statistika_vozaca_po_sezoni WHERE ime_vozaca = 'Max' AND prezime_vozaca = 'Verstappen';
```

Pogled statistika_vozaca_po_sezoni kreiran je s ciljem dobivanja detaljne i objedinjene statistike učinka svakog vozača unutar pojedine sezone. Ovaj view omogućuje analizu rezultata vozača kroz više aspekata, uključujući broj odvoženih utrka, osvojene bodove, pobjede, postolja, kaznene sekunde te prosječno vrijeme po krugu, čime pruža sveobuhvatan uvid u performanse vozača tijekom sezone.

Pogled povezuje podatke iz više tablica kako bi se osigurala potpuna informacija o svakom nastupu vozača. Tablica vozac koristi se za dohvat identiteta vozača, dok se preko tablice vozac_bolid osigurava veza između vozača i njegovih nastupa u sezoni. Tablica sudjelovanje povezuje vozače s konkretnim utrkama, a tablica rezultat sadrži podatke o ostvarenim vremenima i bodovima. Tablice utrka, staza i sezona omogućuju određivanje sezone, broja krugova te konteksta utrke, dok tablica tim osigurava informaciju o timu kojem vozač pripada. Tablica kazna povezana je pomoću LEFT JOIN kako bi se u statistiku uključile kazne ako postoje, ali da se vozači bez kazni i dalje prikazuju u rezultatima.

U okviru pogleda, stupac broj_utrka izračunava se kao broj različitih utrka u kojima je vozač sudjelovao tijekom sezone. Stupac ukupni_bodovi predstavlja zbroj svih bodova koje je vozač osvojio u toj sezoni. Broj pobjeda računa se kao broj utrka u kojima je vozač osvojio točno 25 bodova, dok se broj postolja određuje brojem utrka u kojima je vozač osvojio najmanje 15 bodova. Ukupne kaznene sekunde dobivaju se zbrajanjem svih kazni dodijeljenih vozaču tijekom sezone.

Posebno važan dio pogleda je izračun prosječnog vremena po krugu, koji se dobiva pretvaranjem ukupnog vremena utrke u sekunde te dijeljenjem s brojem krugova na stazi, nakon čega se računa prosjek za sve utrke u sezoni. Time se omogućuje usporedba tempa vozača neovisno o duljini pojedine staze.

Podaci su grupirani po vozaču, sezoni i timu, čime se osigurava da svaki red u pogledu predstavlja jednog vozača u jednoj sezoni. Rezultati su sortirani silazno prema ukupnom broju bodova, a zatim prema broju pobjeda, što omogućuje brz pregled najuspješnijih vozača u sezoni. Dodatni SELECT upiti omogućuju dohvat kompletne statistike svih vozača ili filtriranje rezultata za određenog vozača.

6.3 Statistički podaci o sustavu

```
CREATE OR REPLACE VIEW view_statistika AS
SELECT
    (SELECT COUNT(*) FROM tim) AS broj_timova,
    (SELECT COUNT(*) FROM vozac) AS broj_vozaca,
    (SELECT AVG(budzet) FROM tim) AS prosjecni_budzet,
    (SELECT naziv FROM tim ORDER BY budzet DESC LIMIT 1) AS najbogatiji_tim;
```

Pogled view_statistika kreiran je s ciljem prikaza sažetih statističkih podataka o sustavu u jednoj tablici. Ovaj pogled ne prikazuje pojedinačne zapise, već agregirane informacije koje pružaju opći pregled stanja timova i vozača.

U pogledu se izračunava ukupan broj timova u sustavu korištenjem podupita nad tablicom tim, kao i ukupan broj vozača pomoću podupita nad tablicom vozac. Također se izračunava prosječan budžet timova, čime se dobiva uvid u financijsko stanje ekipa u prosjeku. Posljednji stupac prikazuje naziv tima s najvećim budžetom, koji se određuje sortiranjem timova prema iznosu budžeta silazno i odabirom prvog zapisa.

6.4 Statistika timova

```sql
CREATE OR REPLACE VIEW view_statistika_timova AS
SELECT
    t.id AS id_tim,
    t.naziv AS tim,
    COUNT(DISTINCT v.id) AS broj_vozaca,
    t.budzet AS ukupni_budzet,
    AVG(TIMESTAMPDIFF(YEAR, v.datum_rodjenja, CURDATE())) AS prosjecna_starost_vozaca,
    COUNT(DISTINCT u.id_sponzor) AS broj_sponzora
FROM tim t
LEFT JOIN vozac v ON v.id_tim = t.id
LEFT JOIN ugovor u ON u.id_tim = t.id
GROUP BY t.id, t.naziv, t.budzet;

select * from view_statistika_timova;
```

Pogled view_statistika_timova kreiran je s ciljem prikaza objedinjene statistike za svaki tim. Ovaj pogled omogućuje analizu osnovnih podataka o timu, njegovim vozačima i sponzorima u jedinstvenom prikazu.

U pogledu se za svaki tim prikazuje identifikacijski broj i naziv tima, ukupan broj vozača koji su povezani s timom, trenutni budžet tima te prosječna starost vozača. Prosječna starost izračunava se na temelju razlike između datuma rođenja vozača i trenutnog datuma. Također se izračunava broj sponzora povezanih s timom na temelju sklopljenih ugovora.

Korištenjem LEFT JOIN naredbi osigurava se da u prikazu budu uključeni i timovi koji trenutno nemaju vozače ili sponzore. Grupiranje podataka po timu omogućuje dobivanje sažetih informacija za svaki tim.

6.5 Ukupni broj bodova za timove

```
CREATE OR REPLACE VIEW view_bodovi_timova AS
SELECT
    t.id AS id_tim,
    t.naziv AS tim,
    SUM(p.bodovi) AS ukupni_bodovi
FROM poredak p
JOIN vozac v ON v.id = p.id_vozac
JOIN tim t ON t.id = v.id_tim
GROUP BY t.id, t.naziv;

select * from view_bodovi_timova;
```

Pogled view_bodovi_timova služi za prikaz ukupnog broja bodova koje su timovi osvojili tijekom sezone. Ovaj pogled omogućuje jednostavan uvid u uspješnost svakog tima na temelju ostvarenih rezultata njegovih vozača.

U pogledu se povezuju tablice poredak, vozac i tim, čime se osigurava da se bodovi pravilno pripišu odgovarajućem timu putem njegovih vozača. Za svaki tim izračunava se zbroj bodova svih vozača koji mu pripadaju, a podaci su grupirani po identifikacijskom broju i nazivu tima.

6.6 Najbrži vozači

```
CREATE OR REPLACE VIEW view_najbrzi_vozaci AS
SELECT
    r.id_utrka,
    vb.id_vozac,
    MIN(r.vrijeme) AS najbrze_vrijeme
FROM rezultat r
JOIN vozac_bolid vb ON vb.id = r.id_vozac_bolid
GROUP BY r.id_utrka, vb.id_vozac;


select * from view_najbrzi_vozaci;
```

Pogled view_najbrzi_vozaci kreiran je radi prikaza najbržeg ostvarenog vremena svakog vozača na pojedinoj utrci. Ovaj pogled omogućuje analizu brzine i performansi vozača po utrkama.

U pogledu se povezuju tablice rezultat i vozac_bolid, čime se osigurava veza između rezultata i vozača koji je sudjelovao u utrci. Za svaku kombinaciju utrke i vozača izračunava se najbrže ostvareno vrijeme korištenjem agregatne funkcije MIN. Podaci su grupirani po identifikacijskom broju utrke i vozača, što omogućuje da se za svakog vozača prikaže njegovo najbolje vrijeme na određenoj utrci.

6.7 Broj utrka po sezoni

```
CREATE OR REPLACE VIEW view_broj_utrka_sezona AS
SELECT
    s.godina,
    COUNT(u.id) AS broj_utrka
FROM sezona s
LEFT JOIN utrka u ON u.id_sezona = s.id
GROUP BY s.godina;


select * from view_broj_utrka_sezona;
```

Pogled view_broj_utrka_sezona služi za prikaz broja utrka po pojedinoj sezoni. Ovaj pogled omogućuje jednostavan uvid u opseg natjecanja unutar svake sezone.

U pogledu se povezuju tablice sezona i utrka, pri čemu se za svaku sezonu broji ukupan broj pripadajućih utrka. Korištenjem LEFT JOIN naredbe osigurava se da budu uključene i sezone koje nemaju evidentirane utrke. Podaci su grupirani prema godini sezone, čime se dobiva sažeti pregled broja utrka po godinama.

6.8 Prosječna starost vozača

```sql
CREATE OR REPLACE VIEW view_prosjecna_starost_timova AS
SELECT
    t.id AS id_tim,
    t.naziv AS tim,
    ROUND(AVG(TIMESTAMPDIFF(YEAR, v.datum_rodjenja, CURDATE())),2) AS prosjecna_starost
FROM tim t
JOIN vozac v ON v.id_tim = t.id
GROUP BY t.id, t.naziv;

SELECT tim, prosjecna_starost
FROM view_prosjecna_starost_timova
WHERE id_tim = 1;
```

Pogled view_prosjecna_starost_timova kreiran je s ciljem izračuna prosječne starosti vozača za svaki tim. Ovaj pogled omogućuje jednostavan uvid u dobnu strukturu vozača unutar pojedinog tima.

U pogledu se povezuju tablice tim i vozac te se za svaki tim izračunava prosječna starost vozača na temelju razlike između datuma rođenja i trenutnog datuma. Rezultat se zaokružuje na dvije decimale radi preglednijeg prikaza. Podaci su grupirani po identifikacijskom broju i nazivu tima.

6.9 Analiza uspješnosti timova po budžetu

```sql
DROP VIEW IF EXISTS ucinkovitost_timova;

CREATE VIEW ucinkovitost_timova AS
SELECT tim.id AS id_tim, tim.naziv AS tim, sezona.id AS id_sezona, sezona.godina, tim.budzet,
        SUM(rezultat.bodovi) AS ukupni_bodovi,
        SUM(
            CASE
                WHEN rezultat.bodovi = 25 THEN 1
                ELSE 0
            END
        ) AS pobjede,
        ROUND(
            SUM(rezultat.bodovi) / NULLIF(tim.budzet / 1000000, 0),
            2
        ) AS bodovi_po_milijunu
    FROM tim
        JOIN vozac ON vozac.id_tim = tim.id
        JOIN vozac_bolid ON vozac_bolid.id_vozac = vozac.id
        JOIN rezultat ON rezultat.id_vozac_bolid = vozac_bolid.id
        JOIN utrka ON utrka.id = rezultat.id_utrka
        JOIN sezona ON sezona.id = utrka.id_sezona

        GROUP BY tim.id, tim.naziv, sezona.id, sezona.godina, tim.budzet;

SELECT *
    FROM ucinkovitost_timova
        WHERE godina = 2025
        ORDER BY pobjede DESC;
```

Pogled ucinkovitost_timova kreiran je radi analize uspješnosti timova u odnosu na raspoloživi budžet tijekom pojedine sezone. Ovaj pogled omogućuje usporedbu sportskih rezultata i financijskih resursa timova.

U pogledu se povezuju tablice tim, vozac, vozac_bolid, rezultat, utrka i sezona kako bi se za svaki tim i sezonu prikazali ukupni osvojeni bodovi, broj pobjeda te iznos budžeta. Broj pobjeda izračunava se prebrojavanjem rezultata s maksimalnim brojem bodova, dok se učinkovitost tima dodatno izražava kroz pokazatelj bodova po milijunu budžeta.

Izračun bodova po milijunu omogućuje realniju usporedbu timova različite financijske snage, budući da stavlja ostvarene bodove u odnos s uloženim sredstvima.

6.10 Poredak utrke

```
CREATE VIEW poredak_utrke AS
SELECT utrka.id AS id_utrka, utrka.naziv AS utrka, vozac.id AS id_vozac, vozac.ime, vozac.prezime,
       tim.naziv AS tim, rezultat.vrijeme, rezultat.bodovi,
       COUNT(rezultat2.id) + 1 AS pozicija, IFNULL(SUM(kazna.sekunde), 0) AS kaznene_sekunde
    FROM rezultat
        JOIN utrka ON utrka.id = rezultat.id_utrka
        JOIN vozac_bolid ON vozac_bolid.id = rezultat.id_vozac_bolid
        JOIN vozac ON vozac.id = vozac_bolid.id_vozac
        JOIN tim ON tim.id = vozac.id_tim

    LEFT JOIN rezultat AS rezultat2
        ON rezultat2.id_utrka = rezultat.id_utrka
        AND rezultat2.vrijeme < rezultat.vrijeme

    LEFT JOIN sudjelovanje
        ON sudjelovanje.id_utrka = utrka.id
       AND sudjelovanje.id_vozac_bolid = vozac_bolid.id

    LEFT JOIN kazna
        ON kazna.id_sudjelovanje = sudjelovanje.id

    GROUP BY
        utrka.id,
        utrka.naziv,
        vozac.id,
        vozac.ime,
        vozac.prezime,
        tim.naziv,
        rezultat.id,
        rezultat.vrijeme,
        rezultat.bodovi;
```

Pogled poredak_utrke služi za prikaz detaljnog poretka vozača po pojedinoj utrci. Ovaj pogled omogućuje cjelovit pregled rezultata utrke, uključujući ostvarena vremena, bodove, konačne pozicije i kaznene sekunde.

U pogledu se povezuju tablice rezultat, utrka, vozac_bolid, vozac i tim kako bi se za svaku utrku i vozača prikazali osnovni podaci o nastupu. Pozicija vozača izračunava se usporedbom vremena s ostalim vozačima u istoj utrci, pri čemu se koristi dodatno povezivanje tablice rezultat sa samom sobom. Broj zapisa s boljim vremenom određuje konačnu poziciju vozača.

Dodatno se u pogledu zbrajaju kaznene sekunde povezane s vozačem i utrkom, čime se omogućuje uvid u utjecaj kazni na konačni poredak.

6.11 Vozači po najvećim brojem kaznenih bodova u sezoni

```sql
DROP VIEW IF EXISTS top_vozaci_po_kaznama;

CREATE VIEW top_vozaci_po_kaznama AS
SELECT vozac.id AS id_vozac, vozac.ime, vozac.prezime, tim.naziv AS tim, COUNT(kazna.id) AS broj_kazni,
    IFNULL(SUM(kazna.sekunde),0) AS ukupno_sekundi, utrka.id_sezona
    FROM vozac
        JOIN tim ON tim.id = vozac.id_tim
        JOIN vozac_bolid ON vozac_bolid.id_vozac = vozac.id
        JOIN sudjelovanje ON sudjelovanje.id_vozac_bolid = vozac_bolid.id
        JOIN utrka ON utrka.id = sudjelovanje.id_utrka
        LEFT JOIN kazna ON kazna.id_sudjelovanje = sudjelovanje.id
    GROUP BY vozac.id, tim.naziv, utrka.id_sezona;

SELECT *
    FROM top_vozaci_po_kaznama
    WHERE id_sezona = 1
    ORDER BY broj_kazni DESC, ukupno_sekundi DESC
    LIMIT 3;
```

Pogled top_vozaci_po_kaznama služi za prikaz vozača s najvećim brojem kazni i najvećim ukupnim brojem kaznenih sekundi u određenoj sezoni. Cilj ovog pogleda je analiza discipline vozača tijekom natjecanja.

U pogledu se povezuju tablice vozac, tim, vozac_bolid, sudjelovanje, utrka i kazna kako bi se osigurala potpuna veza između vozača, timova, njihovih nastupa u utrkama i izrečenih kazni. Za svakog vozača i sezonu izračunava se ukupan broj kazni pomoću agregatne funkcije COUNT, kao i zbroj kaznenih sekundi pomoću funkcije SUM. Funkcija IFNULL koristi se kako bi se u slučaju da vozač nema kazni prikazala vrijednost nula.

Podaci su grupirani po identifikacijskom broju vozača, nazivu tima i sezoni, čime se omogućuje usporedba vozača unutar iste sezone. Pogled se može dodatno filtrirati po sezoni te sortirati prema broju kazni i ukupnim kaznenim sekundama, čime se dobiva rang-lista vozača s najizraženijim prekršajima.

6.12 Broj utrka u državama

```
-- 5) VIEW: sezona u državama (broj utrka u državi)
DROP VIEW IF EXISTS sezona_u_drzavama;

CREATE VIEW sezona_u_drzavama AS
SELECT utrka.id_sezona, staza.drzava, COUNT(utrka.id) AS broj_utrka
    FROM utrka
    JOIN staza ON staza.id = utrka.id_staza
    GROUP BY utrka.id_sezona, staza.drzava;

SELECT drzava, broj_utrka
    FROM sezona_u_drzavama
    WHERE id_sezona = 1
    ORDER BY broj_utrka DESC, drzava;
```

Pogled sezona_u_drzavama služi za prikaz broja utrka koje su se održale u pojedinoj državi tijekom određene sezone. Ovaj pogled omogućuje analizu geografske raspodjele utrka unutar sezone.

U pogledu se povezuju tablice utrka i staza, pri čemu se za svaku kombinaciju sezone i države izračunava broj održanih utrka. Podaci su grupirani prema identifikacijskom broju sezone i državi u kojoj se staza nalazi, čime se dobiva pregled koliko je utrka voženo u svakoj državi tijekom sezone.

7. PROCEDURE

7.1. Izračunavanje poretka sezone

```
DELIMITER //

CREATE PROCEDURE izracunaj_poredak_sezone (IN p_sezona INT)
BEGIN
    START TRANSACTION;

    DELETE FROM poredak
    WHERE id_sezona = p_sezona;

    INSERT INTO poredak (id_vozac, id_sezona, bodovi, pobjede, postolja, mjesto)
    SELECT
        id_vozac,
        id_sezona,
        bodovi,
        pobjede,
        postolja,
        mjesto
    FROM view_poredak_sezone
    WHERE id_sezona = p_sezona;

    COMMIT;
END//
```

Procedura **izracunaj_poredak_sezone** kreirana je s ciljem automatskog izračuna i ažuriranja poretka vozača za odabranu sezonu na temelju podataka dobivenih iz pogleda view_poredak_sezone. Procedura prima jedan ulazni parametar p_sezona, koji predstavlja identifikacijski broj sezone za koju se poredak izračunava. Na početku izvođenja procedure pokreće se transakcija (START TRANSACTION) kako bi se osiguralo da se sve promjene nad podacima izvrše kao jedna cjelina te da ne dođe do djelomičnog ažuriranja baze u slučaju pogreške.

U prvom koraku procedura briše sve postojeće zapise iz tablice poredak koji se odnose na zadanu sezonu, čime se osigurava da u tablici ne ostanu zastarjeli ili duplicirani podaci. Nakon toga slijedi umetanje novih podataka u tablicu poredak, pri čemu se koriste rezultati iz pogleda view_poredak_sezone. U tom koraku za svakog vozača u zadanoj sezoni preuzimaju se identifikacijski broj vozača, identifikacijski broj sezone, ukupan broj bodova, broj pobjeda, broj postolja te mjesto u poretku, koji su već prethodno izračunati unutar samog pogleda.

Na kraju procedure izvršava se potvrda transakcije (COMMIT), čime se sve promjene trajno spremaju u bazu podataka. Ovakav pristup omogućuje sigurno i konzistentno ažuriranje poretka sezone te osigurava da tablica poredak uvijek sadrži aktualne i točne podatke temeljene na rezultatima utrka. Procedura se može pozivati više puta za različite sezone te predstavlja centralni mehanizam za održavanje sezonskog poretka unutar sustava.

7.2. Najbrži vozač tima

```
DROP PROCEDURE IF EXISTS najbrzi_vozac_tima_u_utrci;

DELIMITER //
CREATE PROCEDURE najbrzi_vozac_tima_u_utrci(IN p_id_tim INT,IN p_id_utrka INT)
BEGIN
    SELECT vozac.id AS vozac_id,
           vozac.ime AS ime,
           vozac.prezime AS prezime,
           rezultat.vrijeme AS najbolje_vrijeme
    FROM rezultat
    JOIN vozac_bolid ON rezultat.id_vozac_bolid = vozac_bolid.id
    JOIN vozac ON vozac_bolid.id_vozac = vozac.id
    JOIN tim ON vozac.id_tim = tim.id
    WHERE tim.id = p_id_tim
      AND rezultat.id_utrka = p_id_utrka
    ORDER BY rezultat.vrijeme ASC
    LIMIT 1;
END //
DELIMITER ;

CALL najbrzi_vozac_tima_u_utrci(1, 5);
```

Procedura najbrzi_vozac_tima_u_utrci kreirana je s ciljem automatskog pronalaska najbržeg vozača određenog tima na konkretnoj utrci. Procedura prima dva ulazna parametra: p_id_tim, koji predstavlja identifikacijski broj tima, te p_id_utrka, koji označava identifikacijski broj utrke za koju se traži najbrži vozač. Izvođenjem procedure vraćaju se podaci o vozaču koji je ostvario najbolje vrijeme unutar zadanog tima na odabranoj utrci.

Unutar procedure koristi se SELECT upit koji povezuje tablice rezultat, vozac_bolid, vozac i tim. Tablica rezultat sadrži vremena ostvarena na utrci, dok se preko tablice vozac_bolid osigurava ispravna veza između rezultata i konkretnog vozača. Tablica vozac koristi se za dohvat osobnih podataka vozača, a tablica tim za filtriranje vozača koji pripadaju zadanom timu. Uvjeti u WHERE dijelu upita ograničavaju rezultate isključivo na vozače određenog tima (tim.id = p_id_tim) i na rezultate iz konkretne utrke (rezultat.id_utrka = p_id_utrka).

Rezultati se sortiraju prema vremenu vožnje uzlazno (ORDER BY rezultat.vrijeme ASC), čime se osigurava da se na vrhu liste nalazi vozač s najkraćim vremenom. Korištenjem ograničenja LIMIT 1 vraća se samo jedan zapis, odnosno najbrži vozač tog tima na zadanoj utrci. Povratni rezultat procedure uključuje identifikacijski broj vozača, njegovo ime i prezime te ostvareno najbolje vrijeme.

Poziv procedure CALL najbrzi_vozac_tima_u_utrci(1, 5); vraća podatke o najbržem vozaču tima s identifikatorom 1 na utrci s identifikatorom 5, čime se omogućuje jednostavna i brza analiza performansi timskih vozača po utrkama.

7.3. Popis naziva staza

```
DROP PROCEDURE IF EXISTS imena_staza_u_sezoni;
DELIMITER //

CREATE PROCEDURE imena_staza_u_sezoni (
    IN p_id_sezona INT,
    OUT p_staze TEXT
)
BEGIN
    DECLARE v_naziv VARCHAR(100);
    DECLARE gotovo BOOLEAN DEFAULT FALSE;

    DECLARE cur_staze CURSOR FOR
        SELECT DISTINCT s.naziv
        FROM utrka u
        JOIN staza s ON s.id = u.id_staza
        WHERE u.id_sezona = p_id_sezona
        ORDER BY s.naziv;

    DECLARE CONTINUE HANDLER FOR NOT FOUND
        SET gotovo = TRUE;

    SET p_staze = '';
    OPEN cur_staze;

    petlja: LOOP
        FETCH cur_staze INTO v_naziv;

        IF gotovo THEN
            LEAVE petlja;
        END IF;

       IF p_staze = '' THEN
           SET p_staze = v_naziv;
       ELSE
           SET p_staze = CONCAT(p_staze, '; ', v_naziv);
       END IF;
    END LOOP;

    CLOSE cur_staze;
END//

DELIMITER ;
```

Procedura imena_staza_u_sezoni služi za dohvat svih staza na kojima su se utrke održale tijekom određene sezone te za njihovo objedinjavanje u jedan tekstualni zapis. Kao ulazni parametar prima identifikacijski broj sezone, dok se rezultat vraća putem izlaznog parametra u obliku popisa naziva staza razdvojenih točka-zarezom.

Unutar procedure koristi se kursor koji dohvaća jedinstvene nazive staza povezane s utrkama u zadanoj sezoni, pri čemu se podaci preuzimaju iz tablica utrka i staza. Korištenjem ključne riječi DISTINCT osigurava se da se svaka staza u popisu pojavi samo jednom, čak i ako je na istoj stazi održano više utrka. Podaci su dodatno sortirani po nazivu staze kako bi rezultat bio pregledan i čitljiv.

Tijekom izvođenja petlje svaki dohvaćeni naziv staze postupno se dodaje u izlaznu varijablu. Uvjetom se provjerava radi li se o prvom zapisu ili o nastavku niza, čime se osigurava ispravno formatiranje izlaznog teksta.

7.4. Dodavanje utrke

```
DELIMITER //

CREATE PROCEDURE dodaj_utrku (
    p_naziv VARCHAR(100),
    p_datum DATE,
    p_id_sezona INT,
    p_id_staza INT
)
BEGIN
    INSERT INTO utrka (naziv, datum, id_sezona, id_staza)
    VALUES (p_naziv, p_datum, p_id_sezona, p_id_staza);
END//

DELIMITER ;


CALL dodaj_utrku('Najjaca utrka', '2025-12-15', 1, 1);
select * from utrka;
```

Procedura dodaj_utrku služi za jednostavno i kontrolirano dodavanje nove utrke u sustav. Kao ulazne parametre prima naziv utrke, datum održavanja, identifikacijski broj sezone te identifikacijski broj staze na kojoj se utrka održava.

Unutar procedure izvršava se umetanje novog zapisa u tablicu utrka korištenjem proslijeđenih parametara. Na ovaj način osigurava se standardizirani način unosa utrka te se smanjuje mogućnost pogrešaka pri ručnom unosu podataka.

7.5. Povećanje broja pobjeda vozaču

```
DELIMITER //

CREATE PROCEDURE dodaj_pobjedu_pobjedniku (IN p_id_utrka INT)
BEGIN
    DECLARE v_id_vozac INT DEFAULT NULL;
    DECLARE v_id_sezona INT;

    SELECT id_sezona
    INTO v_id_sezona
    FROM utrka
    WHERE id = p_id_utrka;

    SELECT vozac_bolid.id_vozac
    INTO v_id_vozac
    FROM rezultat
    JOIN vozac_bolid  ON rezultat.id_vozac_bolid = vozac_bolid.id
    WHERE rezultat.id_utrka = p_id_utrka
    ORDER BY rezultat.bodovi DESC
    LIMIT 1;

    IF v_id_vozac IS NOT NULL THEN
        UPDATE poredak
        SET pobjede = pobjede + 1
        WHERE id_vozac = v_id_vozac
          AND id_sezona = v_id_sezona;
    END IF;

END //

DELIMITER ;
CALL dodaj_pobjedu_pobjedniku(5);
```

Procedura dodaj_pobjedu_pobjedniku služi za automatsko povećanje broja pobjeda vozaču koji je ostvario prvo mjesto na određenoj utrci. Kao ulazni parametar prima identifikacijski broj utrke za koju se pobjeda evidentira.

Unutar procedure najprije se iz tablice utrka dohvaća identifikacijski broj sezone u kojoj se utrka održava. Zatim se iz tablica rezultat i vozac_bolid određuje vozač koji je ostvario najbolji rezultat na toj utrci, pri čemu se kao kriterij koristi najveći broj osvojenih bodova. Dobiveni identifikator vozača pohranjuje se u lokalnu varijablu.

Ako je pobjednik uspješno pronađen, procedura ažurira tablicu poredak tako da se tom vozaču poveća broj pobjeda u odgovarajućoj sezoni. Time se osigurava da se statistika pobjeda automatski ažurira na temelju rezultata utrke.

7.6. Prosječna kazna po utrkama za tim

```
DROP PROCEDURE IF EXISTS prosjek_kazni_tima;

DELIMITER //

CREATE PROCEDURE prosjek_kazni_tima(
    IN p_id_tim INT,
    IN p_id_sezona INT,
    OUT p_broj_utrka INT,
    OUT p_ukupne_kazne DECIMAL(10,2),
    OUT p_prosjek DECIMAL(8,2)
)
BEGIN
    SELECT
        COUNT(DISTINCT utrka.id),
        IFNULL(SUM(kazna.sekunde), 0)
    INTO
        p_broj_utrka,
        p_ukupne_kazne
    FROM tim
        JOIN vozac ON vozac.id_tim = tim.id
        JOIN vozac_bolid ON vozac_bolid.id_vozac = vozac.id
        JOIN sudjelovanje ON sudjelovanje.id_vozac_bolid = vozac_bolid.id
        JOIN utrka ON utrka.id = sudjelovanje.id_utrka
        LEFT JOIN kazna ON kazna.id_sudjelovanje = sudjelovanje.id
    WHERE tim.id = p_id_tim
      AND utrka.id_sezona = p_id_sezona;


    IF p_broj_utrka = 0 THEN
        SET p_prosjek = 0;
    ELSE
        SET p_prosjek = ROUND(p_ukupne_kazne / p_broj_utrka, 2);
    END IF;
END//

DELIMITER ;
```

Procedura prosjek_kazni_tima služi za izračun kazni određenog tima u odabranoj sezoni, pri čemu rezultate vraća putem izlaznih parametara. Kao ulazne parametre prima identifikacijski broj tima i identifikacijski broj sezone, dok kao izlazne parametre vraća broj utrka, ukupni broj kaznenih sekundi te prosječnu kaznu po utrci.

Unutar procedure povezuju se tablice tim, vozac, vozac_bolid, sudjelovanje, utrka i kazna kako bi se obuhvatili svi nastupi vozača tog tima u zadanoj sezoni. Najprije se izračunava ukupan broj različitih utrka u kojima je tim sudjelovao te zbroj svih kaznenih sekundi ostvarenih u toj sezoni. Dobivene vrijednosti spremaju se u izlazne parametre p_broj_utrka i p_ukupne_kazne.

Nakon toga se provodi provjera broja utrka. Ako tim nije sudjelovao ni u jednoj utrci u zadanoj sezoni, prosječna kazna postavlja se na nulu kako bi se izbjeglo dijeljenje s nulom. U suprotnom slučaju, prosječna kazna po utrci računa se dijeljenjem ukupnog broja kaznenih sekundi s brojem utrka te se rezultat zaokružuje na dvije decimale i sprema u izlazni parametar p_prosjek.

8. FUNKCIJE

8.1. Bodovi vozača u sezoni

```
DROP FUNCTION IF EXISTS bodovi_vozaca_sezona;

DELIMITER //
CREATE FUNCTION bodovi_vozaca_sezona(p_id_vozac INT,p_id_sezona INT)
RETURNS INT
DETERMINISTIC
BEGIN
    DECLARE ukupno INT;
    SELECT SUM(rezultat.bodovi)
    INTO ukupno
    FROM rezultat
    JOIN vozac_bolid ON rezultat.id_vozac_bolid = vozac_bolid.id
    JOIN utrka ON rezultat.id_utrka = utrka.id
    WHERE vozac_bolid.id_vozac = p_id_vozac
      AND utrka.id_sezona = p_id_sezona;
    RETURN IFNULL(ukupno, 0);
END //
DELIMITER ;

SELECT bodovi_vozaca_sezona(7, 1);
```

Funkcija bodovi_vozaca_sezona kreirana je s ciljem izračuna ukupnog broja bodova koje je određeni vozač osvojio u jednoj sezoni. Funkcija prima dva ulazna parametra: p_id_vozac, koji predstavlja identifikacijski broj vozača, te p_id_sezona, koji označava identifikacijski broj sezone za koju se bodovi računaju. Povratna vrijednost funkcije je cijeli broj (INT) koji predstavlja ukupan zbroj bodova vozača u zadanoj sezoni.

Unutar funkcije deklarira se lokalna varijabla ukupno, u koju se pohranjuje rezultat izračuna. Izračun se provodi pomoću agregacijske funkcije SUM nad stupcem bodovi iz tablice rezultat. Kako bi se osigurala ispravna povezanost podataka, tablica rezultat povezuje se s tablicom vozac_bolid, čime se rezultati vežu uz točno određenog vozača, te s tablicom utrka, kako bi se omogućilo filtriranje rezultata prema sezoni. Uvjetima u WHERE dijelu upita ograničava se izračun samo na zapise koji pripadaju zadanom vozaču i odabranoj sezoni.

Funkcija koristi izraz IFNULL(ukupno, 0) kako bi se u slučaju da vozač nema nijedan rezultat u zadanoj sezoni vratila vrijednost 0, umjesto NULL. Time se osigurava konzistentan i predvidljiv rezultat funkcije. Ova funkcija može se koristiti unutar SELECT upita, procedura ili view-ova, primjerice za prikaz ukupnih bodova vozača u sezoni ili za usporedbu učinka više vozača.

Poziv funkcije SELECT bodovi_vozaca_sezona(7, 1); vraća ukupan broj bodova koje je vozač s identifikatorom 7 osvojio u sezoni s identifikatorom 1.

8.2. Pretvaranje brzine

```
DELIMITER //

CREATE FUNCTION kmh_u_mph(brzina_kmh DECIMAL(6,2))
RETURNS DECIMAL(6,2)
DETERMINISTIC
BEGIN
    RETURN brzina_kmh * 0.621371;
END//

DELIMITER ;

SELECT kmh_u_mph(100);
```

Funkcija kmh_u_mph služi za pretvorbu brzine iz kilometara na sat u milje na sat. Kao ulazni parametar prima vrijednost brzine izražene u kilometrima na sat, dok kao rezultat vraća izračunatu vrijednost u miljama na sat.

Pretvorba se provodi množenjem ulazne vrijednosti s konstantom 0.621371, koja predstavlja omjer između kilometara i milja. Funkcija je označena kao deterministička, što znači da za iste ulazne vrijednosti uvijek vraća isti rezultat.

8.3. Sudjelovanja vozača u utrkama

```
DELIMITER //

CREATE FUNCTION broj_ucestvovanja_vozaca(p_id_vozac INT)
RETURNS INT
DETERMINISTIC
BEGIN
    DECLARE br INT;

    SELECT COUNT(DISTINCT s.id)
    INTO br
    FROM vozac_bolid vb
    JOIN sudjelovanje s ON s.id_vozac_bolid = vb.id
    JOIN rezultat r ON r.id_utrka = s.id_utrka
    WHERE vb.id_vozac = p_id_vozac;

    RETURN br;
END//

DELIMITER ;

SELECT broj_ucestvovanja_vozaca(1);
```

Funkcija broj_ucestvovanja_vozaca služi za izračun broja utrka u kojima je određeni vozač sudjelovao. Kao ulazni parametar prima identifikacijski broj vozača, a kao rezultat vraća ukupan broj njegovih nastupa.

Funkcija dohvaća podatke iz tablica vozac_bolid, sudjelovanje i rezultat te broji jedinstvene zapise sudjelovanja koji su povezani s danim vozačem. Korištenjem funkcije COUNT(DISTINCT ...) osigurava se da se svaka utrka broji samo jednom, čak i u slučaju složenijih veza između tablica.

8.4. Vozač s najvećim brojem nastupa

```
DELIMITER //
CREATE FUNCTION najbolji_vozac_po_nastupima(
    p_id_tim INT
)
RETURNS INT
DETERMINISTIC
BEGIN
    DECLARE vozac_id INT;

    SELECT vb.id_vozac
    INTO vozac_id
    FROM vozac_bolid vb
    JOIN vozac v ON v.id = vb.id_vozac
    JOIN sudjelovanje s ON s.id_vozac_bolid = vb.id
    WHERE v.id_tim = p_id_tim
    GROUP BY vb.id_vozac
    ORDER BY COUNT(s.id) DESC
    LIMIT 1;

    RETURN vozac_id;
END//
DELIMITER ;

SELECT v.ime, v.prezime
FROM vozac v
WHERE v.id = najbolji_vozac_po_nastupima(1);

CREATE OR REPLACE VIEW view_bodovi_vozaca AS
SELECT
    vb.id_vozac,
    u.id_sezona,
    COUNT(r.id) * 10 + COUNT(r.vrijeme) * 15 AS bodovi
FROM vozac_bolid vb
JOIN rezultat r ON r.id_vozac_bolid = vb.id
JOIN utrka u ON u.id = r.id_utrka
GROUP BY vb.id_vozac, u.id_sezona;

SELECT * FROM view_bodovi_vozaca;
```

Funkcija najbolji_vozac_po_nastupima služi za određivanje vozača s najvećim brojem nastupa unutar određenog tima. Kao ulazni parametar prima identifikacijski broj tima, a kao rezultat vraća identifikacijski broj vozača koji je najčešće sudjelovao u utrkama za taj tim.

Funkcija povezuje tablice vozac_bolid, vozac i sudjelovanje te filtrira zapise prema odabranom timu. Podaci se grupiraju po vozaču, a zatim se sortiraju prema broju nastupa silazno. Kao rezultat se vraća vozač s najvećim brojem sudjelovanja. Ova funkcija omogućuje analizu pouzdanosti i kontinuirane prisutnosti vozača unutar tima.

Pogled view_bodovi_vozaca kreiran je za izračun ukupnih bodova vozača po sezoni prema prilagođenoj bodovnoj formuli. U pogledu se za svakog vozača i sezonu izračunava broj bodova

na temelju broja zapisa u tablici rezultat, pri čemu se koristi kombinacija dva faktora: broj nastupa i broj zabilježenih vremena. Rezultati su grupirani po vozaču i sezoni, što omogućuje jednostavan pregled bodovnog učinka vozača kroz sezone.

8.5. Najbolji vozač u timu

```
DROP FUNCTION IF EXISTS najbolji_vozac_u_timu;

DELIMITER //

CREATE FUNCTION najbolji_vozac_u_timu(
    p_id_tim INT,
    p_id_sezona INT
)
RETURNS INT
DETERMINISTIC
BEGIN
    DECLARE vozac_id INT;

    SELECT vbv.id_vozac
    INTO vozac_id
    FROM view_bodovi_vozaca vbv
    JOIN vozac v ON v.id = vbv.id_vozac
    WHERE v.id_tim = p_id_tim
      AND vbv.id_sezona = p_id_sezona
    ORDER BY vbv.bodovi DESC
    LIMIT 1;

    RETURN vozac_id;
END//

DELIMITER ;

SELECT CONCAT(v.ime, ' ', v.prezime) AS najbolji_vozac
FROM vozac v
WHERE v.id = najbolji_vozac_u_timu(1,1);
```

Funkcija najbolji_vozac_u_timu služi za određivanje najboljeg vozača unutar određenog tima u odabranoj sezoni. Kao ulazne parametre prima identifikacijski broj tima i identifikacijski broj sezone, a kao rezultat vraća identifikacijski broj vozača s najboljim ostvarenim rezultatom.

Funkcija koristi pogled view_bodovi_vozaca, koji sadrži izračunate bodove vozača po sezoni, te ga povezuje s tablicom vozac kako bi se osiguralo filtriranje prema timu. Podaci se dodatno filtriraju prema odabranoj sezoni i sortiraju prema broju bodova silazno, čime se osigurava da se kao rezultat odabere vozač s najvećim brojem bodova.

8.6. Prosječne kaznene sekunde vozača po utrci u sezoni

```
DROP FUNCTION IF EXISTS prosjek_kazni_po_utrci_vozaca;

DELIMITER //

CREATE FUNCTION prosjek_kazni_po_utrci_vozaca(p_id_vozac INT, p_id_sezona INT) RETURNS DECIMAL(8,2)
DETERMINISTIC
BEGIN
    DECLARE ukupno_kazni DECIMAL(10,2);
    DECLARE nastupi INT;

    SELECT SUM(kazna.sekunde) INTO ukupno_kazni
        FROM kazna
            JOIN sudjelovanje ON sudjelovanje.id = kazna.id_sudjelovanje
            JOIN vozac_bolid ON vozac_bolid.id = sudjelovanje.id_vozac_bolid
            JOIN utrka ON utrka.id = sudjelovanje.id_utrka
        WHERE vozac_bolid.id_vozac = p_id_vozac AND utrka.id_sezona = p_id_sezona;

    SELECT COUNT(DISTINCT sudjelovanje.id_utrka) INTO nastupi
    FROM vozac_bolid
        JOIN sudjelovanje ON sudjelovanje.id_vozac_bolid = vozac_bolid.id
        JOIN utrka ON utrka.id = sudjelovanje.id_utrka
    WHERE vozac_bolid.id_vozac = p_id_vozac AND utrka.id_sezona = p_id_sezona;

    IF nastupi = 0 THEN
        RETURN 0;
    END IF;

    RETURN ROUND(COALESCE(ukupno_kazni, 0) / nastupi, 2);


END//

DELIMITER ;

SELECT prosjek_kazni_po_utrci_vozaca(2,1);
select * from kazna;
```

Funkcija prosjek_kazni_po_utrci_vozaca služi za izračun prosječnog broja kaznenih sekundi koje vozač dobiva po utrci u određenoj sezoni. Kao ulazne parametre prima identifikacijski broj vozača i identifikacijski broj sezone, a kao rezultat vraća decimalnu vrijednost prosjeka kaznenih sekundi.

Funkcija najprije izračunava ukupan broj kaznenih sekundi za odabranog vozača u zadanoj sezoni, pri čemu povezuje tablice kazna, sudjelovanje, vozac_bolid i utrka. Zatim se izračunava broj utrka u kojima je vozač sudjelovao tijekom te sezone. Ako vozač nema nijedan nastup u sezoni, funkcija vraća vrijednost nula kako bi se izbjeglo dijeljenje s nulom. Funkcija dijeli ukupan broj kaznenih sekundi s brojem nastupa te rezultat zaokružuje na dvije decimale.

8.7. Postotak sprint utrka u sezoni za vozača

```
DELIMITER //

CREATE FUNCTION postotak_sprint_utrka_vozaca (p_id_vozac INT, p_id_sezona INT) RETURNS DECIMAL(5,2)
DETERMINISTIC
BEGIN
    DECLARE ukupno INT;
    DECLARE sprint INT;

    SELECT COUNT(DISTINCT sudjelovanje.id_utrka) INTO ukupno
    FROM vozac_bolid
        JOIN sudjelovanje ON sudjelovanje.id_vozac_bolid = vozac_bolid.id
        JOIN utrka ON utrka.id = sudjelovanje.id_utrka
    WHERE vozac_bolid.id_vozac = p_id_vozac AND utrka.id_sezona = p_id_sezona;

    SELECT COUNT(DISTINCT sudjelovanje.id_utrka) INTO sprint
    FROM vozac_bolid
        JOIN sudjelovanje ON sudjelovanje.id_vozac_bolid = vozac_bolid.id
        JOIN utrka ON utrka.id = sudjelovanje.id_utrka
    WHERE vozac_bolid.id_vozac = p_id_vozac AND utrka.id_sezona = p_id_sezona AND utrka.naziv LIKE '%Sprint%';

    IF ukupno = 0 THEN
        RETURN 0;
    END IF;

    RETURN ROUND((sprint / ukupno) * 100, 2);
END//

DELIMITER ;
```

Funkcija postotak_sprint_utrka_vozaca služi za izračun postotka sprint utrka u kojima je određeni vozač sudjelovao tijekom odabrane sezone. Kao ulazne parametre prima identifikacijski broj vozača i identifikacijski broj sezone, a kao rezultat vraća postotnu vrijednost.

Funkcija najprije izračunava ukupan broj utrka u kojima je vozač sudjelovao tijekom sezone, a zatim broj onih utrka koje su označene kao sprint utrke, pri čemu se sprint utrke prepoznaju

prema nazivu utrke. Ako vozač nema nijedan nastup u sezoni, funkcija vraća vrijednost nula kako bi se izbjeglo dijeljenje s nulom.

Na kraju se izračunava postotak sprint utrka u odnosu na ukupan broj nastupa te se rezultat zaokružuje na dvije decimale.

9. OKIDAČI

9.1. Sprječavanje promjene naziva tima

```
DELIMITER //
CREATE TRIGGER bu_naziv
    BEFORE UPDATE ON tim
    FOR EACH ROW
BEGIN

    DECLARE cur_s INT DEFAULT 0;

    IF NEW.naziv != OLD.naziv THEN
        SELECT count(*)
            INTO cur_s
            FROM sezona
            WHERE sezona.godina = YEAR(CURDATE())
            AND MONTH(CURDATE()) BETWEEN 3 AND 12;

        IF cur_s > 0 THEN
            SIGNAL SQLSTATE '40000'
            SET MESSAGE_TEXT = 'Zabranjeno mjenjanje imena';
        END IF;
    END IF;

END//
DELIMITER ;
```

Okidač bu_naziv definiran je kao BEFORE UPDATE okidač nad tablicom tim te služi za sprječavanje promjene naziva tima tijekom trajanja aktivne sezone. Njegova osnovna svrha je očuvanje konzistentnosti i povijesne točnosti podataka, budući da promjena imena tima usred sezone može dovesti do neusklađenosti u rezultatima, poretku i statistici.

Okidač se aktivira prije svake izmjene zapisa u tablici tim. Unutar okidača deklarira se lokalna varijabla cur_s, koja se koristi za provjeru postojanja aktivne sezone u tekućoj godini. Prvo se provjerava je li došlo do promjene naziva tima, usporedbom stare (OLD.naziv) i nove (NEW.naziv) vrijednosti. Ako naziv nije promijenjen, okidač se ne izvršava dalje.

U slučaju da se naziv tima pokušava promijeniti, okidač provodi upit nad tablicom sezona kojim se provjerava postoji li sezona u tekućoj kalendarskoj godini i nalazi li se trenutni datum unutar razdoblja trajanja sezone, koje je ovdje definirano kao razdoblje od ožujka do prosinca. Rezultat tog upita sprema se u varijablu cur_s.

Ako se utvrdi da sezona postoji i da je u tijeku (cur_s > 0), okidač prekida izvršavanje naredbe UPDATE te generira grešku pomoću naredbe SIGNAL SQLSTATE. Time se sprječava promjena naziva tima i korisniku se vraća poruka o pogrešci „Zabranjeno mijenjanje imena“. Na taj način osigurava se da se naziv tima ne može mijenjati dok sezona traje, dok su promjene dopuštene izvan sezone.

9.2. Automatsko dodavanje kazne na vrijeme

```
DELIMITER //
CREATE TRIGGER ai_vrijeme
    AFTER INSERT ON kazna
    FOR EACH ROW
BEGIN
    DECLARE trka_id INT;
    DECLARE vozac_id INT;

    SELECT id_utrka, id_vozac_bolid
        INTO trka_id, vozac_id
        FROM sudjelovanje
        WHERE id = NEW.id_sudjelovanje;

    UPDATE rezultat
    SET vrijeme = addtime(vrijeme, sec_to_time(NEW.sekunde))
    WHERE id_utrka = trka_id AND id_vozac_bolid = vozac_id;

END//
DELIMITER ;
```

Okidač ai_vrijeme definiran je kao AFTER INSERT okidač nad tablicom kazna te služi za automatsko dodavanje kaznenih sekundi na ukupno vrijeme vozača u utrci. Njegova je svrha osigurati da se svaka izrečena vremenska kazna odmah i dosljedno odrazi na konačni rezultat vozača, bez potrebe za ručnim ažuriranjem tablice rezultat.

Okidač se aktivira nakon umetanja novog zapisa u tablicu kazna. Unutar okidača deklariraju se lokalne varijable trka_id i vozac_id, koje služe za privremeno spremanje identifikatora utrke i kombinacije vozač–bolid na koje se kazna odnosi. Ti se podaci dohvaćaju iz tablice sudjelovanje na temelju identifikacijskog broja sudjelovanja (NEW.id_sudjelovanje) koji je upravo umetnut u tablicu kazna.

Nakon dohvaćanja identifikatora utrke i vozača, okidač ažurira odgovarajući zapis u tablici rezultat. Ažuriranje se provodi tako da se postojećem vremenu vožnje dodaju kaznene sekunde pomoću funkcije ADDTIME, pri čemu se broj kaznenih sekundi (NEW.sekunde) najprije pretvara u vremenski format pomoću funkcije SEC_TO_TIME. Uvjet u WHERE dijelu UPDATE naredbe osigurava da se promjena primijeni isključivo na rezultat vozača na konkretnoj utrci na koju se kazna odnosi.

9.3. Automatsko dodavanje bodova

```
DELIMITER //
CREATE TRIGGER au_poredak
    AFTER UPDATE ON utrka
    FOR EACH ROW

BEGIN

    DECLARE kraj INT DEFAULT 0;
    DECLARE pozicija INT;
    DECLARE bod_add INT;
    DECLARE vozac_id INT;
    DECLARE sezona_id INT;
    DECLARE vb_id INT;

    DECLARE cur CURSOR FOR
        SELECT id_vozac_bolid
            FROM rezultat
            WHERE id_utrka = NEW.id
            ORDER BY vrijeme ASC, id ASC;
    DECLARE CONTINUE HANDLER FOR NOT FOUND SET kraj = 1;

    IF OLD.status = 'Traje' AND NEW.status = 'Zavrsena' THEN
        SELECT id_sezona
            INTO sezona_id
            FROM utrka
            WHERE id = NEW.id;

        SET pozicija = 0;
```

```
OPEN cur;

read_loop: LOOP
FETCH cur INTO vb_id;

IF kraj = 1 THEN
    LEAVE read_loop;
END IF;

SET pozicija = pozicija + 1;

SET bod_add = CASE pozicija
    WHEN 1 THEN 25
    WHEN 2 THEN 18
    WHEN 3 THEN 15
    WHEN 4 THEN 12
    WHEN 5 THEN 10
    WHEN 6 THEN 8
    WHEN 7 THEN 6
    WHEN 8 THEN 4
    WHEN 9 THEN 2
    WHEN 10 THEN 1
    ELSE 0
END;
```

```
        IF pozicija = 1 THEN
            UPDATE poredak
                SET pobjede = pobjede + 1
                WHERE id_vozac = vozac_id
                AND id_sezona = sezona_id;
        END IF;

        IF pozicija <= 3 THEN
            UPDATE poredak
                SET postolja = postolja + 1
                WHERE id_vozac = vozac_id
                AND id_sezona = sezona_id;
        END IF;

        IF bod_add > 0 THEN
            SELECT id_vozac
                INTO vozac_id
                FROM vozac_bolid
                WHERE id = vb_id;

            UPDATE poredak
                SET bodovi = bodovi + bod_add
                WHERE id_vozac = vozac_id AND id_sezona = sezona_id;
        END IF;
        END LOOP;
        CLOSE cur;
    END IF;
END//
DELIMITER ;
```

Okidač au_poredak služi za automatsku dodjelu bodova, pobjeda i postolja vozačima nakon završetka utrke. Okidač se aktivira nakon izmjene zapisa u tablici utrka te se izvršava isključivo u slučaju kada se status utrke promijeni iz Traje u Zavrsena, čime se osigurava da se bodovanje provede samo jednom, po završetku utrke.

Nakon završetka utrke dohvaćaju se svi rezultati za tu utrku, sortirani prema ostvarenim vremenima, čime se dobiva konačni poredak vozača. Za prvih deset vozača automatski se dodjeljuju bodovi prema unaprijed definiranoj bodovnoj shemi, dok se za pobjednika dodatno evidentira pobjeda, a za prva tri vozača postolje. Svi se podaci ažuriraju u tablici poredak za odgovarajuću sezonu, čime se osigurava da poredak vozača uvijek odražava službene rezultate utrke.

9.4. Kontrola budžeta tima pri unosu novog troška

```
DELIMITER //
CREATE TRIGGER bi_trosak
    BEFORE INSERT ON trosak_tima
    FOR EACH ROW
BEGIN

    DECLARE budg DECIMAL(15,2);

    SELECT budzet
        INTO budg
        FROM tim
        WHERE id = NEW.id_tim;

    IF NEW.iznos > budg THEN
        SIGNAL SQLSTATE '40000'
        SET MESSAGE_TEXT = 'Nema dovoljno sredstava u budžetu';
    ELSE
        UPDATE tim
            SET budzet = budzet - NEW.iznos
            WHERE id = NEW.id_tim;
    END IF;

END //
DELIMITER ;
```

Okidač bi_trosak definiran je kao BEFORE INSERT okidač nad tablicom trosak_tima te služi za kontrolu budžeta tima prilikom unosa novog troška. Njegova je svrha spriječiti unos troškova koji bi premašili raspoloživi budžet tima te osigurati automatsko ažuriranje budžeta nakon uspješnog unosa troška.

Prije umetanja novog zapisa u tablicu trosak_tima, okidač dohvaća trenutni iznos budžeta pripadajućeg tima iz tablice tim. Ako je iznos novog troška veći od raspoloživog budžeta, okidač prekida izvršavanje naredbe umetanja i generira pogrešku, čime se sprječava prekoračenje budžeta. U suprotnom slučaju, ako tim ima dovoljno sredstava, budžet tima se automatski umanjuje za iznos unesenog troška.

9.5. Provjera dobi vozača

```
DELIMITER //
CREATE TRIGGER bi_datum_rodenja
    BEFORE INSERT ON vozac
    FOR EACH ROW
BEGIN

    IF NEW.datum_rodjenja > date_sub(curdate(), interval 18 year) THEN
        SIGNAL SQLSTATE '40000'
        SET MESSAGE_TEXT = 'Vozač premlad';
    END IF;

END//
DELIMITER ;
```

Okidač bi_datum_rodenja definiran je kao BEFORE INSERT okidač nad tablicom vozac te služi za provjeru dobi vozača prilikom unosa novog zapisa. Njegova je svrha osigurati da se u sustav mogu unositi samo vozači koji ispunjavaju minimalni dobni uvjet.

Prije umetanja zapisa u tablicu vozac, okidač provjerava je li datum rođenja novog vozača stariji od osamnaest godina u odnosu na trenutni datum. Ako je utvrđeno da vozač u trenutku unosa nema navršenih 18 godina, okidač prekida izvršavanje naredbe umetanja i generira pogrešku s odgovarajućom porukom. Time se sprječava unos nevažećih podataka te se osigurava poštivanje pravila o minimalnoj dobi vozača unutar sustava.

9.6. Praćenje budžeta tima

```
DELIMITER //
CREATE TRIGGER au_tim
AFTER UPDATE ON tim
FOR EACH ROW
BEGIN
    IF OLD.budzet <> NEW.budzet THEN
        INSERT INTO sistemski_log (tablica, akcija, poruka)
        VALUES ('tim', 'UPDATE', CONCAT('Budžet tima ', OLD.naziv, ' promijenjen sa ',
               OLD.budzet, ' na ', NEW.budzet));
    END IF;
END;
//
DELIMITER ;
```

Okidač au_tim definiran je kao AFTER UPDATE okidač nad tablicom tim te služi za praćenje promjena budžeta timova. Njegova je svrha evidentirati svaku izmjenu budžeta u posebnoj tablici za logiranje, čime se osigurava transparentnost i mogućnost naknadne provjere financijskih promjena.

Nakon svake izmjene zapisa u tablici tim, okidač provjerava je li došlo do promjene vrijednosti stupca budzet usporedbom stare (OLD.budzet) i nove (NEW.budzet) vrijednosti. Ako je promjena utvrđena, u tablicu sistemski_log umeće se novi zapis koji sadrži naziv tablice na kojoj je promjena izvršena, vrstu akcije te opisnu poruku s informacijom o staroj i novoj vrijednosti budžeta tima.

9.7. Provjera prijave vozača

```
DROP TRIGGER IF EXISTS bi_provjera_sudjelovanja;

DELIMITER //

CREATE TRIGGER bi_provjera_sudjelovanja
BEFORE INSERT ON rezultat
FOR EACH ROW
BEGIN
    DECLARE postoji INT;

    SELECT COUNT(*) INTO postoji
    FROM sudjelovanje
    WHERE id_utrka = NEW.id_utrka AND id_vozac_bolid = NEW.id_vozac_bolid;

    IF postoji = 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Vozač nije prijavljen za ovu utrku';
    END IF;
END//

DELIMITER ;


INSERT INTO rezultat (id_utrka, id_vozac_bolid, vrijeme, bodovi)
VALUES (67, 1, '01:40:00', 5);
```

Okidač bi_provjera_sudjelovanja definiran je kao BEFORE INSERT okidač nad tablicom rezultat te služi za provjeru je li vozač prijavljen za određenu utrku prije unosa rezultata. Njegova je svrha osigurati logičku ispravnost i konzistentnost podataka u bazi.

Prije umetanja novog zapisa u tablicu rezultat, okidač provjerava postoji li zapis u tablici sudjelovanje koji povezuje odabranu utrku i kombinaciju vozač–bolid. Rezultat provjere sprema se u lokalnu varijablu postoji.

Ako se utvrdi da vozač nije prijavljen za navedenu utrku, okidač prekida izvršavanje naredbe umetanja i generira poruku o pogrešci.

10. TRANSAKCIJE

10.1 Unos podataka o pobjedniku i ažuriranje poretka

```
DELIMITER //

CREATE PROCEDURE upisi_pobjednika (
    IN p_utrka INT,
    IN p_vozac_bolid INT,
    IN p_vrijeme TIME
)
BEGIN
    DECLARE v_id_vozac INT;
    DECLARE v_id_sezona INT;

    START TRANSACTION;

    SELECT id_sezona INTO v_id_sezona
    FROM utrka WHERE id = p_utrka;

    SELECT id_vozac INTO v_id_vozac
    FROM vozac_bolid WHERE id = p_vozac_bolid;

    INSERT INTO rezultat (id_utrka, id_vozac_bolid, vrijeme, bodovi)
    VALUES (p_utrka, p_vozac_bolid, p_vrijeme, 25);

    UPDATE poredak
    SET bodovi = bodovi + 25,
        pobjede = pobjede + 1
    WHERE id_vozac = v_id_vozac
      AND id_sezona = v_id_sezona;

    COMMIT;
END//
```

Procedura upisi_pobjednika predstavlja transakciju koja služi za unos podataka o pobjedniku određene utrke te za istovremeno ažuriranje poretka vozača u pripadajućoj sezoni. Time se osigurava da se unos rezultata i ažuriranje statistike provedu kao jedna cjelina.

Na početku procedure pokreće se transakcija kako bi se zajamčila konzistentnost podataka. Iz tablice utrka dohvaća se identifikacijski broj sezone za odabranu utrku, dok se iz tablice vozac_bolid dohvaća identifikacijski broj vozača koji je ostvario pobjedu. Nakon toga se u tablicu rezultat umeće novi zapis koji bilježi pobjednika utrke s dodijeljenih 25 bodova.

U sljedećem koraku ažurira se tablica poredak, pri čemu se pobjedniku povećava ukupan broj bodova i broj pobjeda za odgovarajuću sezonu. Na kraju se transakcija potvrđuje naredbom COMMIT, čime se osigurava da se sve promjene trajno zabilježe u bazi podataka.

10.2 Brisanje bodova za zasluženu kaznu

```
START TRANSACTION;

INSERT INTO kazna (id_sudjelovanje, opis, tip_kazne, sekunde)
VALUES (11, 'Opasna vožnja', 'Time Penalty', 10.0);

SET @kazna_id = LAST_INSERT_ID();

UPDATE poredak
SET id_kazna = @kazna_id,
    bodovi = bodovi - 5
WHERE id_vozac = (
    SELECT vb.id_vozac
    FROM sudjelovanje s
    JOIN vozac_bolid vb ON s.id_vozac_bolid = vb.id
    WHERE s.id = 11
)
AND id_sezona = 1;

COMMIT;
```

Ova transakcija služi za evidentiranje kazne vozaču te za istovremeno umanjenje njegovih bodova u poretku sezone. Cilj transakcije je osigurati da se unos kazne i oduzimanje bodova provedu kao jedinstvena i konzistentna cjelina.

Na početku se pokreće transakcija, zatim se u tablicu kazna umeće novi zapis koji opisuje prekršaj, vrstu kazne i kaznene sekunde za određeno sudjelovanje u utrci. Nakon unosa kazne dohvaća se identifikacijski broj upravo umetnutog zapisa pomoću funkcije LAST_INSERT_ID, kako bi se ta kazna mogla povezati s poretkom vozača.

U sljedećem koraku ažurira se tablica poredak tako da se vozaču kojem je izrečena kazna pridruži identifikator kazne te mu se umanji ukupan broj bodova za fiksnih pet bodova u odgovarajućoj sezoni. Identifikacijski broj vozača dohvaća se putem podupita koji povezuje

tablice sudjelovanje i vozac_bolid. Na kraju se transakcija potvrđuje naredbom COMMIT, čime se osigurava da se sve promjene trajno zapišu u bazu podataka.

10.3 Promjena stanja bolida

```
START TRANSACTION;
UPDATE bolid
SET stanje = 'Na servisu'
WHERE id = (
    SELECT id_bolid
    FROM vozac_bolid
    WHERE id_vozac = 2 AND id_sezona = 1
);

INSERT INTO vozac_bolid (id_bolid, id_vozac, id_sezona)
VALUES (3, 2, 1);

COMMIT;
select * from bolid;
select * from vozac_bolid;
```

Ova transakcija služi za evidentiranje promjene tehničkog stanja bolida te za dodjelu novog bolida vozaču unutar iste sezone. Cilj transakcije je osigurati da se obje promjene provedu istovremeno i konzistentno.

Na početku transakcije mijenja se stanje postojećeg bolida vozača u vrijednost Na servisu. Identifikator bolida dohvaća se putem podupita iz tablice vozac_bolid na temelju identifikacijskog broja vozača i sezone. Time se bilježi da bolid više nije dostupan za natjecanje.

U sljedećem koraku u tablicu vozac_bolid umeće se novi zapis kojim se vozaču dodjeljuje novi bolid za istu sezonu. Na taj način vozač i dalje može sudjelovati u utrkama unatoč servisu prethodnog bolida.

Na kraju se transakcija potvrđuje naredbom COMMIT, čime se osigurava da su obje promjene trajno zabilježene u bazi podataka.

10.4 Evidentiranje prelaska vozača u drugi tim

```
DELIMITER //

CREATE PROCEDURE transfer_vozaca (
    IN p_vozac INT,
    IN p_novi_tim INT,
    IN p_novi_bolid INT,
    IN p_sezona INT
)
BEGIN
    START TRANSACTION;

    UPDATE vozac
    SET id_tim = p_novi_tim
    WHERE id = p_vozac;

    DELETE FROM vozac_bolid WHERE id_vozac = p_vozac;

    INSERT INTO vozac_bolid(id_bolid, id_vozac, id_sezona)
    VALUES (p_novi_bolid, p_vozac, p_sezona);

    COMMIT;
END //

DELIMITER ;
DROP PROCEDURE transfer_vozaca;
SELECT * FROM vozac;
CALL transfer_vozaca(10, 2, 2,1 );
SELECT * FROM vozac_bolid;
```

Procedura transfer_vozaca služi za evidentiranje prelaska vozača u drugi tim te za ažuriranje pripadajućih podataka o bolidu u okviru iste sezone. Procedura kombinira poslovnu logiku i transakciju kako bi se osigurala konzistentnost podataka tijekom prijenosa vozača.

Na početku procedure pokreće se transakcija. Najprije se u tablici vozac ažurira identifikator tima, čime se bilježi novi tim kojem vozač pripada. Zatim se iz tablice vozac_bolid brišu svi postojeći zapisi vezani uz tog vozača, čime se uklanja prethodna veza između vozača i bolida.

U sljedećem koraku umeće se novi zapis u tablicu vozac_bolid kojim se vozaču dodjeljuje novi bolid za odabranu sezonu. Nakon uspješnog izvršenja svih naredbi, transakcija se potvrđuje naredbom COMMIT, čime se osigurava da se sve promjene trajno zapišu u bazu podataka.

10.5 Poništavanje plaćanja ako tim prekorači budžet

```
DELIMITER //

CREATE PROCEDURE dodaj_trosak_timu (
    IN p_tim INT,
    IN p_sezona YEAR,
    IN p_kategorija VARCHAR(100),
    IN p_iznos DECIMAL(15,2)
)
BEGIN
    DECLARE v_budzet DECIMAL(15,2);

    START TRANSACTION;
    SELECT budzet INTO v_budzet FROM tim WHERE id = p_tim FOR UPDATE;

    IF v_budzet < p_iznos THEN
        ROLLBACK;
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Nedovoljno stanje budžeta!';
    ELSE
        UPDATE tim
        SET budzet = budzet - p_iznos
        WHERE id = p_tim;
        INSERT INTO trosak_tima(id_tim, sezona, kategorija, iznos)
        VALUES (p_tim, p_sezona, p_kategorija, p_iznos);

        COMMIT;
    END IF;
END //

DELIMITER ;
```

Procedura dodaj_trosak_timu služi za kontrolirani unos troškova tima uz istovremenu provjeru raspoloživog budžeta. Procedura osigurava da se trošak evidentira samo ako tim ima dovoljno financijskih sredstava, čime se sprječava prekoračenje budžeta.

Na početku procedure pokreće se transakcija te se dohvaća trenutni budžet odabranog tima uz zaključavanje zapisa kako bi se spriječile paralelne izmjene. Ako je iznos novog troška veći od dostupnog budžeta, transakcija se poništava naredbom ROLLBACK te se generira poruka o pogrešci, čime se sprječava unos nevažećih podataka.

U slučaju da tim ima dovoljno sredstava, budžet tima se umanjuje za iznos troška, a zatim se u tablicu trosak_tima umeće novi zapis s podacima o trošku. Nakon uspješnog izvršenja svih naredbi, transakcija se potvrđuje naredbom COMMIT.

11. INICIJALIZACIJA I KONFIGURACIJA

Flask(__name__): kreiranje instance Flask aplikacije.
app.secret_key: postavljanje tajnog ključa za sesiju i kolačiće

12. RUTE I FUNKCIONALNOSTI

Glavna ruta / : povratak na početnu stranicu.

Ruta /login : prijava u sustav.
Ruta /logut : odjava iz sustava.

Ruta /vozaci : lista svih vozače.
Ruta /obrisi-vozaca/<int:id>' : brisanje vozača po idju.

Ruta /utrke : popis utrka.
Ruta /dodaj_utrku : moguće dodati utrku.
Ruta /promijeni-status-utrke : moguće promijeniti status utrka.

Ruta /promijeni-budzet : mijenja budzet ekipi.

Ruta /transfer : upis transfera.
Ruta /izvrsi-transfer : izvršava opisani transfer.

Ruta /financije : prati financijsko stanje
Ruta /dodaj-trosak : prati troškove

Ruta /statistika : statistika timova i vozača

13. INTERAKCIJA S BAZOM

cursor: izvođenje SQL upita.
fetchall(): dohvaćanje svih redaka SQL upita
fetchone(): dohvaćanje jednog retka SQL upita
commit(): spremanje podataka u bazu
close(): zatvaranje veza s bazom

14. PORUKE I REDIREKCIJA

flash() : ispis poruke korisniku
redirect(url_for('ime_rute')): preusmjeravanje na drugu rutu nakon akcije